Code Bharat Frontend Interview Guide

Structured interview revision edition for Akamai preparation

Built from your original guide, with the full content retained and reorganized for faster revision.

How to use this file

- Read the 'Quick Interview Revision' section first. It gives you the answer flow you can speak in an interview even if you do not remember every file immediately.
- Use the later file-by-file sections to refresh implementation details. Those sections retain your original detailed content, but the formatting is improved so code and notes are easier to scan.
- Whenever a recruiter asks 'how did you implement this?', answer in three layers: purpose of the feature, main frontend flow, and the exact state/API logic used.

Quick Interview Revision

1. 45-second project summary

Code Bharat is a coding-platform frontend built with React and Vite. I used React Router to separate public pages, problem-solving pages, and admin pages. For global state I kept Redux Toolkit only for authentication so the navbar and protected routes can know whether a user is logged in. Most page-specific data like problem lists, examples, code editor state, and admin form rows stay in local component state. The frontend talks to two services: the main backend for auth, problem data, profile, and submissions; and a separate compiler service for running code-related actions.

2. How to explain the architecture

The browser first loads index.html, then main.jsx mounts the React app. App.jsx creates the route tree. Layouts provide shared wrappers like the main navbar, login shell, problem workspace shell, or admin shell. Inside each layout, page components fetch their own data and render the actual UI. This means the app is structured as Entry -> Router -> Layout -> Page -> API calls.

3. Authentication flow

Authentication is cookie-based. On app load, Redux can dispatch fetchUser, which calls /api/auth/me with withCredentials: true. If the backend finds a valid cookie, it returns the current user and Redux stores it in state.auth.user. If the request fails, the user becomes null. This state is then used by the navbar to show the right buttons and by the admin guard to block non-admin users.

4. Why Redux only for auth

I intentionally kept Redux small. Authentication affects many places globally, so it belongs in Redux. But page-specific state like form fields, fetched problems, examples, hidden test cases, editor code, language choice, and tab state are easier to keep local with useState and useEffect. This avoids overengineering and makes each page easier to reason about.

5. Problem-solving flow

The user opens the problem set page, chooses a problem, and the problem page loads the statement and related data. The editor UI is part of the problem workspace. When the user runs or reviews code, the frontend can call the compiler service. When the user submits final code, it sends the submission to the backend, which can then evaluate it against hidden test cases and update user/problem statistics.

6. Admin problem creation flow

Admin creation is a two-step workflow. First the frontend sends the top-level problem data and visible examples to the backend. After that request succeeds, it gets the created problem ID. Then it sends hidden test cases in a second API call to the testcase endpoint. The advantage is simpler backend separation. The tradeoff is that if the second call fails, the problem may already exist without hidden test cases, so this flow is functionally correct but not fully atomic.

7. Why the project uses separate layouts

Different parts of the product need different chrome. Public pages share the main landing-site style. Login and register pages use a simplified auth layout. Problem pages need a focused solving layout. Admin pages need a

protected workspace. Layout separation keeps shared UI reusable and prevents each page from repeating the same wrapper code.

8. Frontend decisions you can defend

I used Vite because it gives fast startup and a simple React setup. I used Tailwind because it lets me style quickly with utility classes and maintain a consistent design system without writing large CSS files. I used Framer Motion for small landing-page animations. I used React Router loaders on problem routes so those pages can fetch route-specific data before rendering the main workspace.

9. Weaknesses and improvements

A good interview answer is to mention honest tradeoffs. In this codebase, data fetching is distributed across pages instead of being centralized with React Query or a service layer. The NotFound page exists but there is no catch-all route, so 404 handling is incomplete. The useAuth hook and Redux auth pattern overlap a bit, so auth access could be standardized. The two-step admin creation flow could be made transactional on the backend. These are sensible future improvements, not failures.

10. Common interviewer questions and answer frames

Q: Why did you use withCredentials? A: Because auth is cookie-based, so the browser must send cookies on cross-origin requests.
Q: How does route protection work? A: The router places admin routes under RequireAdmin; that component checks auth/role before allowing the admin layout to render.
Q: Why not store everything in Redux? A: Only global shared state was worth centralizing; page-local data stays local to reduce complexity.
Q: Why separate backend and compiler URLs? A: The app depends on two services with different responsibilities, so central URL config keeps environment switching clean.
Q: What would you improve first? A: I would standardize the auth pattern, add a real 404 route, and centralize fetching/error handling for better maintainability.

Detailed file-by-file walkthrough (full original content retained)

Everything below comes from your original guide. The content is preserved, but the formatting has been improved so code blocks, tables, and headings are easier to revise before interviews.

Code Bharat Frontend Interview Guide

Generated from this repository on 2026-03-24 14:23

Goal: explain how the frontend is structured, how each page is designed and implemented, how routing/state work, and how UI code talks to the backend.

1. Frontend Overview

This frontend is a React 19 + Vite single-page application styled with Tailwind CSS utilities, animated with Framer Motion, state-managed with Redux Toolkit for auth, and routed with React Router. It has a public experience, an authenticated problem-solving workspace, and a separate admin workspace.

2. Render Flow

```
index.html
  -> src/main.jsx
    -> Redux Provider(store)
      -> App.jsx
        -> React Router tree
          -> Layout
            -> Page
              -> Shared components
                -> axios calls to backend/compiler
```

3. How UI State Travels

1. App boot starts in `main.jsx`, which wraps the whole tree in Redux `Provider`.
2. `App.jsx` builds the browser router and selects layout + page by URL.
3. Layouts provide shared frame elements like navigation, footer, admin sidebar, or page chrome.
4. Pages own most of the screen-specific fetching and local UI state.
5. Shared auth state lives in Redux and is fetched through `fetchUser` thunk.
6. Axios calls use URLs from `src/config/urls.js` and send cookies with `withCredentials: true`.
7. Problem solving also calls the separate compiler service for run/review actions.

4. Route Map

Path	Layout / Guard	Page	Purpose
/	RootLayout	HomePage	Public landing page
/profile	RootLayout	ProfilePage	Authenticated profile experience handled in page logic
/contests	RootLayout	ContestsPage	Contest packs built from

			live problems
/leaderboard	RootLayout	LeaderboardPage	Public ranking page
/explore	RootLayout	ExplorePage	Searchable discovery page
/problems	ProblemLayout	ProblemSetPage	Problem list with loader
/problems/:problemId	ProblemLayout	ProblemPage	Problem workspace with loader
/login	LoginLayout	LoginPage	User login
/register	LoginLayout	RegisterPage	User registration
/admin/login	AdminAuthLayout	AdminLoginPage	Admin login
/admin/register	AdminAuthLayout	AdminRegisterPage	Admin registration
/admin	RequireAdmin -> AdminLayout	AdminDashboardPage	Protected admin dashboard
/admin/problems	RequireAdmin -> AdminLayout	AdminProblemsPage	Protected problem management
/admin/problems/new	RequireAdmin -> AdminLayout	AdminAddProblemPage	Protected problem creation

5. File Groups

Area	Files
Entry	index.html, main.jsx, App.jsx
Config	vite.config.js, eslint.config.js, config/urls.js
State	Redux store, auth slice, auth thunks, auth API
Layouts	RootLayout, ProblemLayout, LoginLayout, AdminAuthLayout, AdminLayout
Middleware	RequireAdmin route guard
Components	Navbar, AuthNavbar, Footer, MeshPageShell, SubmissionTable
Pages	Home, Explore, Contests, Leaderboard, ProblemSet, Problem, Profile, Login, Register, admin pages

6. Environment Variables

Variable	Use
VITE_BACKEND_URL_DEV	Development backend base URL override
VITE_COMPILER_URL_DEV	Development compiler-service URL override

VITE_BACKEND_URL	Production backend base URL
VITE_COMPILER_URL	Production compiler-service base URL

7. Source Tree

```
Frontend/
 - .env
 - .gitignore
 - README.md
 - eslint.config.js
 - index.html
 - package.json
   - App.css
   - App.jsx
     - AuthNavbar.jsx
     - Footer.jsx
     - MeshPageShell.jsx
     - Navbar.jsx
     - SubmissionTable.jsx
     - useAuth.js
     - AdminAuthLayout.jsx
     - AdminLayout.jsx
     - LoginLayout.jsx
     - ProblemLayout.jsx
     - RootLayout.jsx
     - RequireAdmin.jsx
       - AdminAddProblemPage.jsx
       - AdminDashboardPage.jsx
       - AdminLoginPage.jsx
       - AdminProblemsPage.jsx
       - AdminRegisterPage.jsx
     - ContestsPage.jsx
     - ExplorePage.jsx
     - HomePage.jsx
     - LeaderboardPage.jsx
     - LoginPage.jsx
     - NotFoundPage.jsx
     - ProblemPage.jsx
     - ProblemSetPage.jsx
     - ProfilePage.jsx
     - RegisterPage.jsx
       - store.js
         - authAPI.js
         - authSlice.js
         - authThunks.js
     - urls.js
   - index.css
   - main.jsx
 - vite.config.js
```

8. Key Interview Observations

- The app uses Redux only for auth; most page state is local `useState` + `useEffect`.
- Data fetching is distributed across pages instead of centralized in a service layer or React Query.
- The router imports `NotFoundPage`, but there is no catch-all route defined, so the 404 component is currently unused.
- `useAuth` hook exists, but navbar/admin guard rely on Redux thunk fetch instead, so there are two auth-access patterns.
- `App.css` exists but is effectively unused; `index.css` contains the active global styles.
- Problem workspace mixes backend submission calls and compiler-service run/review calls in a single page component.

- Admin problem creation is a two-step workflow: create problem first, then create hidden test cases.

Redacted Frontend `.env`

```
1: VITE_BACKEND_URL=***REDACTED***
2: VITE_COMPILER_URL=***REDACTED***
```

Frontend Package Manifest

File: Frontend/package.json

Purpose: Defines the frontend stack, scripts, runtime mode, and key dependencies.

How it fits into the app: This file decides how the frontend boots, builds, lints, and which libraries are available to the app.

Interview-ready talking points:

- Vite is the build/dev tool, React 19 is the UI runtime, and Tailwind is integrated through the Vite plugin.
- React Router, Redux Toolkit, axios, Monaco, React Split, React Markdown, and Framer Motion are core libraries.

Implementation notes:

- This is a Vite ESM app because `type` is `module`.

Code

```
 1: {
 2:   "name": "frontend",
 3:   "private": true,
 4:   "version": "0.0.0",
 5:   "type": "module",
 6:   "engines": {
 7:     "node": "20.x"
 8:   },
 9:   "scripts": {
10:     "dev": "vite",
11:     "build": "vite build",
12:     "lint": "eslint .",
13:     "preview": "vite preview"
14:   },
15:   "dependencies": {
16:     "@monaco-editor/react": "^4.7.0",
17:     "@reduxjs/toolkit": "^2.8.2",
18:     "@tailwindcss/vite": "^4.1.11",
19:     "axios": "^1.11.0",
20:     "framer-motion": "^12.23.12",
21:     "react": "^19.1.0",
22:     "react-dom": "^19.1.0",
23:     "react-markdown": "^10.1.0",
24:     "react-redux": "^9.2.0",
25:     "react-router-dom": "^7.7.1",
26:     "react-split": "^2.0.14",
27:     "remark-gfm": "^4.0.1"
28:   },
29:   "devDependencies": {
30:     "@eslint/js": "^9.30.1",
31:     "@types/react": "^19.1.8",
32:     "@types/react-dom": "^19.1.6",
33:     "@vitejs/plugin-react": "^4.6.0",
34:     "eslint": "^9.30.1",
35:     "eslint-plugin-react-hooks": "^5.2.0",
36:     "eslint-plugin-react-refresh": "^0.4.20",
37:     "globals": "^16.3.0",
38:     "vite": "^7.0.4"
39:   }
40: }
```

Lines 5-8: Marks the project as ES module based.

Lines 9-14: Defines development, build, lint, and preview commands.

Lines 15-28: Lists runtime libraries used by the UI.

```
Lines 29-40: Lists development tooling like Vite and ESLint.
```

Vite Config

File: Frontend/vite.config.js

Purpose: Registers build-time plugins for React and Tailwind CSS.

How it fits into the app: Used during development and build rather than browser runtime execution.

Interview-ready talking points:

React plugin enables JSX and React-specific Vite behavior.

Tailwind plugin integrates utility-first styling into the build.

Implementation notes:

Config is intentionally minimal.

Code

```
 1: import { defineConfig } from 'vite'
 2: import react from '@vitejs/plugin-react'
 3: import tailwindcss from '@tailwindcss/vite'
 4:
 5: // https://vite.dev/config/
 6: export default defineConfig({
 7:   plugins: [react(),
 8:     tailwindcss(),
 9:   ],
10: })
```

Lines 1-5: Imports Vite config builder and plugins.

Line 6: Starts the Vite config object.

Lines 7-10: Registers React and Tailwind plugins.

ESLint Config

File: Frontend/eslint.config.js

Purpose: Defines lint rules for JavaScript and JSX files.

How it fits into the app: Affects development quality checks rather than runtime logic.

Interview-ready talking points:

Uses flat ESLint config format.

Includes React hooks and React Refresh recommendations.

Implementation notes:

Built `dist` output is globally ignored.

Code

```
 1: import js from '@eslint/js'
 2: import globals from 'globals'
 3: import reactHooks from 'eslint-plugin-react-hooks'
 4: import reactRefresh from 'eslint-plugin-react-refresh'
 5: import { defineConfig, globalIgnores } from 'eslint/config'
 6:
 7: export default defineConfig([
 8:   globalIgnores(['dist']),
 9:   {
10:     files: ['**/*.{js,jsx}'],
11:     extends: [
12:       js.configs.recommended,
13:       reactHooks.configs['recommended-latest'],
14:       reactRefresh.configs.vite,
15:     ],
16:     languageOptions: {
17:       ecmaVersion: 2020,
18:       globals: globals.browser,
19:       parserOptions: {
20:         ecmaVersion: 'latest',
21:         ecmaFeatures: { jsx: true },
22:         sourceType: 'module',
23:       },
24:     },
25:     rules: {
26:       'no-unused-vars': ['error', { varsIgnorePattern: '^[A-Z_]' }],
27:     },
28:   },
29: ])
```

Lines 1-6: Imports ESLint presets and helper packages.

Line 7: Defines the flat ESLint config array.

Lines 8-9: Ignores generated build output.

Lines 10-24: Applies rules to JS and JSX files.

Lines 25-29: Defines project-specific rule overrides.

HTML Shell

File: Frontend/index.html

Purpose: Defines the single HTML page and the root mount target for React.

How it fits into the app: Browser loads this file first and then runs `src/main.jsx`.

Interview-ready talking points:

This is a standard SPA shell with one root div.

Favicon is loaded from a frontend asset.

Code

```
 1: <!doctype html>
 2: <html lang="en">
 3:   <head>
 4:     <meta charset="UTF-8" />
 5:     <link rel="icon" type="image/svg+xml" href="./src/assets/tabLogo.png" />
 6:     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
 7:     <title>Code Bharat</title>
 8:   </head>
 9:   <body>
10:     <div id="root"></div>
11:     <script type="module" src="/src/main.jsx"></script>
12:   </body>
13: </html>
```

Lines 1-4: Declares the HTML document.

Lines 5-9: Sets favicon metadata.

Line 10: Creates the DOM mount point.

Lines 11-13: Starts the React app from `main.jsx`.

React Entrypoint

File: Frontend/src/main.jsx

Purpose: Mounts the app into the DOM and wraps it in Redux Provider.

How it fits into the app: First JavaScript module executed by the browser after `index.html`.

Interview-ready talking points:

Redux Provider makes auth state available to the whole app.

StrictMode helps surface side-effect issues during development.

Implementation notes:

Global styles come from `index.css`.

Code

```
 1: import { StrictMode } from 'react';
 2: import { createRoot } from 'react-dom/client';
 3: import { Provider } from 'react-redux';
 4: import store from './Store/App/store.js';
 5: import App from './App.jsx';
 6: import './index.css';
 7:
 8: createRoot(document.getElementById('root')).render(
 9:   <StrictMode>
10:     <Provider store={store}>
11:       <App />
12:     </Provider>
13:   </StrictMode>
14: );
```

Lines 1-7: Imports React root helpers, Provider, store, App, and global CSS.

Line 8: Mounts the React tree into the DOM.

Line 9: Wraps the app in React StrictMode.

Line 10: Injects the Redux store.

Lines 11-14: Renders the router app.

```
Router Composition
File: Frontend/src/App.jsx
Purpose: Defines the route tree, layouts, loaders, and protected admin section.
How it fits into the app: Decides which layout and page render for every URL in the application.
Interview-ready talking points:
Public, problem workspace, auth, and admin flows are separated with different layouts.
Admin routes are protected with `RequireAdmin` at route level.
Problem pages use React Router loaders.
Implementation notes:
There is no catch-all route, so `NotFoundPage` is currently unused.
```

Code

```
 1: import React from "react";
 2: import {createBrowserRouter,createRoutesFromElements,Route,RouterProvider,} from "react-router-dom";
 3: import RootLayout from "./Layouts/RootLayout.jsx";
 4: import ProblemLayout from "./Layouts/ProblemLayout.jsx";
 5: import LoginLayout from "./Layouts/LoginLayout.jsx";
 6: import HomePage from "./Pages/HomePage.jsx";
 7: import ProfilePage from "./Pages/ProfilePage.jsx";
 8: import ContestsPage from "./Pages/ContestsPage.jsx";
 9: import LeaderboardPage from "./Pages/LeaderboardPage.jsx";
10: import ExplorePage from "./Pages/ExplorePage.jsx";
11: import LoginPage from "./Pages/LoginPage.jsx";
12: import RegisterPage from "./Pages/RegisterPage.jsx";
13: import ProblemSetPage, { problemSetLoader } from "./Pages/ProblemSetPage.jsx";
14: import ProblemPage, { ProblemLoader } from "./Pages/ProblemPage.jsx";
15: import AdminAddProblemPage from "./Pages/Admin/AdminAddProblemPage.jsx";
16: import RequireAdmin from "./Middleware/RequireAdmin.jsx";
17: import AdminLayout from "./Layouts/AdminLayout.jsx";
18: import AdminAuthLayout from "./Layouts/AdminAuthLayout.jsx";
19: import AdminDashboardPage from "./Pages/Admin/AdminDashboardPage.jsx";
20: import AdminProblemsPage from "./Pages/Admin/AdminProblemsPage.jsx";
21: import AdminLoginPage from "./Pages/Admin/AdminLoginPage.jsx";
22: import AdminRegisterPage from "./Pages/Admin/AdminRegisterPage.jsx";
23:
24: function App() {
25:   const router = createBrowserRouter(
26:     createRoutesFromElements(
27:       <>
28:         <Route path="/" element={<RootLayout />}>
29:           <Route index element={<HomePage />} />
30:           <Route path="profile" element={<ProfilePage />} />
31:           <Route path="contests" element={<ContestsPage />} />
32:           <Route path="leaderboard" element={<LeaderboardPage />} />
33:           <Route path="explore" element={<ExplorePage />} />
34:
35:         </Route>
36:
37:         <Route element={<ProblemLayout />}>
38:           <Route
39:             path="/problems"
40:             element={<ProblemSetPage />}
41:             loader={problemSetLoader}
42:           />
43:           <Route
44:             path="/problems/:problemId"
45:             element={<ProblemPage />}
46:             loader={ProblemLoader}
47:           />
48:         </Route>
49:
50:         <Route element={<LoginLayout />}>
51:           <Route path="/login" element={<LoginPage />} />
52:           <Route path="/register" element={<RegisterPage />} />
53:         </Route>
```

```
54:
55:         <Route element={<AdminAuthLayout />}>
56:           <Route path="/admin/login" element={<AdminLoginPage />} />
57:           <Route path="/admin/register" element={<AdminRegisterPage />} />
58:         </Route>
59:
60:         <Route path="/admin" element={<RequireAdmin />}>
61:           <Route element={<AdminLayout />}>
62:             <Route index element={<AdminDashboardPage />} />
63:             <Route path="problems" element={<AdminProblemsPage />} />
64:             <Route path="problems/new" element={<AdminAddProblemPage />} />
65:           </Route>
66:         </Route>
67:       </>
68:     )
69:   );
70:
71:   return <RouterProvider router={router} />;
72: }
73:
74: export default App;
```

Line 24: Defines the top-level app component.

Lines 25-27: Creates the browser router.

Lines 28-36: Defines public root layout routes.

Lines 37-49: Defines problem workspace routes.

Lines 50-54: Defines user auth routes.

Lines 55-59: Defines admin auth routes.

Lines 60-70: Defines protected admin routes.

Lines 71-74: Renders router provider.

```
App CSS
File: Frontend/src/App.css
Purpose: Placeholder stylesheet with no active content.
How it fits into the app: Does not currently affect rendering because it is empty and unused.
Interview-ready talking points:
This file is effectively unused in the current frontend.
Implementation notes:
Shared styling actually lives in `src/index.css` and Tailwind classes.
```

Code

```
1:
```

Global CSS

File: Frontend/src/index.css

Purpose: Loads Tailwind and defines split-pane gutter styling.

How it fits into the app: Imported once in `main.jsx` and applied globally.

Interview-ready talking points:

Tailwind is enabled here with `@import`.

Custom gutter styles support `react-split` in the coding workspace.

Code

```
 1: @import "tailwindcss";
 2:
 3: .gutter {
 4:   background-color: #2d2d2d;
 5:   background-clip: padding-box;
 6: }
 7:
 8: .gutter.gutter-horizontal {
 9:   cursor: col-resize;
10:   width: 6px;
11: }
12:
13: .gutter.gutter-vertical {
14:   cursor: row-resize;
15:   height: 6px;
16: }
```

Lines 1-2: Loads Tailwind CSS.

Lines 3-7: Defines base split-gutter styling.

Lines 8-12: Styles horizontal resize gutter.

Lines 13-16: Styles vertical resize gutter.

URL Config

File: Frontend/src/config/urls.js

Purpose: Computes backend and compiler-service base URLs from Vite environment variables.

How it fits into the app: Imported before axios calls are made from pages and API helpers.

Interview-ready talking points:

Frontend talks to two services: backend API and compiler service.

Development and production URLs are resolved separately.

Code

```
 1: const localBackendUrl =
 2:   import.meta.env.VITE_BACKEND_URL_DEV || "http://localhost:8000";
 3:
 4: const localCompilerUrl =
 5:   import.meta.env.VITE_COMPILER_URL_DEV || "http://localhost:7000";
 6:
 7: export const BACKEND_URL = import.meta.env.DEV
 8:   ? localBackendUrl
 9:   : import.meta.env.VITE_BACKEND_URL || localBackendUrl;
10:
11: export const COMPILER_URL = import.meta.env.DEV
12:   ? localCompilerUrl
13:   : import.meta.env.VITE_COMPILER_URL || localCompilerUrl;
```

Lines 1-3: Defines development backend default or override.

Lines 4-6: Defines development compiler default or override.

Lines 7-10: Chooses backend URL by environment.

Lines 11-13: Chooses compiler URL by environment.

useAuth Hook

File: Frontend/src/Hooks/useAuth.js

Purpose: Provides a small hook for fetching current auth state from the backend.

How it fits into the app: Could be used by any component needing auth, although Redux auth is more common in this repo.

Interview-ready talking points:

Uses local state for `user` and `loading`.

Relies on backend cookie auth with `withCredentials: true`.

Implementation notes:

Appears unused in the current codebase.

Code

```
 1: import { useEffect, useState } from 'react';
 2: import axios from 'axios';
 3: import { BACKEND_URL } from '../config/urls.js';
 4:
 5: export default function useAuth() {
 6:   const [user, setUser] = useState(null);
 7:   const [loading, setLoading] = useState(true);
 8:
 9:   useEffect(() => {
10:     (async () => {
11:       try {
12:         const res = await axios.get(
13:           `${BACKEND_URL}/api/auth/me`,
14:           { withCredentials: true }
15:         );
16:         setUser(res.data.user);
17:       } catch {
18:         setUser(null);
19:       } finally {
20:         setLoading(false);
21:       }
22:     })();
23:   }, []);
24:
25:   return { user, loading };
26: }
```

Line 5: Defines the auth hook.

Lines 6-8: Stores local auth user.

Lines 9-11: Fetches current user on mount.

Lines 12-24: Calls `/api/auth/me`.

Lines 25-26: Returns hook state.

Redux Store

File: Frontend/src/Store/App/store.js

Purpose: Creates the global Redux store.

How it fits into the app: Provided at app boot and read by navbar, admin guard, and other UI pieces.

Interview-ready talking points:

Redux scope is intentionally narrow: only auth is global.

Auth state is exposed under `state.auth`.

Code

```
 1: import { configureStore } from '@reduxjs/toolkit';
 2: import authReducer from '../Features/Auth/authSlice.js';
 3:
 4: export const store = configureStore({
 5:     reducer: {
 6:         auth: authReducer,
 7:     },
 8: });
 9:
10: export default store;
11:
```

Line 4: Creates the Redux store.

Line 5: Registers reducers.

Lines 6-11: Mounts auth reducer.

Auth API Helpers

File: Frontend/src/Store/Features/Auth/authAPI.js

Purpose: Wraps auth-related axios requests used by thunks.

How it fits into the app: Thunk layer calls these helpers when UI needs current-user or logout operations.

Interview-ready talking points:

Separates raw HTTP calls from Redux thunk declarations.

Both calls send cookies with `withCredentials: true`.

Code

```
 1: import axios from "axios";
 2: import { BACKEND_URL } from "../../../config/urls.js";
 3:
 4: export const getUser = async () => {
 5:   const res = await axios.get(`${BACKEND_URL}/api/auth/me`, {
 6:     withCredentials: true,
 7:   });
 8:   return res.data.user;
 9: };
10:
11: export const logout = async () => {
12:   await axios.post(
13:     `${BACKEND_URL}/api/auth/logout`,
14:     {},
15:     { withCredentials: true }
16:   );
17:   return null;
18: };
```

Line 4: Defines current-user request helper.

Lines 5-10: Calls backend current-user endpoint.

Line 11: Defines logout request helper.

Lines 12-18: Calls backend logout endpoint.

Auth Slice

File: Frontend/src/Store/Features/Auth/authSlice.js

Purpose: Stores authenticated user data and loading state in Redux.

How it fits into the app: Global auth state drives navbar rendering and admin route protection.

Interview-ready talking points:

Async state is handled through `extraReducers`.

Rejected auth fetch clears the user.

Code

```
 1: import { createSlice } from "@reduxjs/toolkit";
 2: import { fetchUser, logoutThunk } from "./authThunks";
 3:
 4: const initialState = {
 5:   user: null,
 6:   loading: true,
 7: };
 8:
 9: const authSlice = createSlice({
10:   name: 'auth',
11:   initialState,
12:   reducers: {},
13:   extraReducers: (builder) => {
14:     builder
15:       .addCase(fetchUser.pending, (state) => {
16:         state.loading = true;
17:       })
18:
19:       .addCase(fetchUser.fulfilled, (state, action) => {
20:         state.user = action.payload;
21:         state.loading = false;
22:       })
23:
24:       .addCase(fetchUser.rejected, (state) => {
25:         state.user = null;
26:         state.loading = false;
27:       })
28:
29:       .addCase(logoutThunk.fulfilled, (state) => {
30:         state.user = null;
31:         state.loading = false;
32:       })
33:
34:       .addCase(logoutThunk.rejected, (state) => {
35:         state.user = null;
36:         state.loading = false;
37:       });
38:   },
39: });
40:
41: export default authSlice.reducer;
```

Lines 4-8: Defines default auth state.

Lines 9-14: Creates the auth slice.

Lines 15-18: Marks loading while auth check is in progress.

Lines 19-23: Stores current user on success.

Lines 24-28: Clears user on failed auth check.

Lines 29-41: Clears user on logout.

Auth Thunks

File: Frontend/src/Store/Features/Auth/authThunks.js

Purpose: Defines async thunks for fetching current user and logging out.

How it fits into the app: Components dispatch these thunks to keep auth state synchronized with backend.

Interview-ready talking points:

`fetchUser` hydrates auth state for navbar and guards.

`logoutThunk` keeps Redux state aligned with server logout.

Code

```
1: import { createAsyncThunk } from "@reduxjs/toolkit";
2: import { getUser, logout } from './authAPI';
3:
4: export const fetchUser = createAsyncThunk('/auth/fetchUser', getUser);
5:
6: export const logoutThunk = createAsyncThunk('/auth/logout', logout);
```

Lines 1-3: Imports thunk helper.

Lines 4-5: Defines current-user thunk.

Line 6: Defines logout thunk.

Admin Dashboard Page

File: Frontend/src/Pages/Admin/AdminDashboardPage.jsx

Purpose: Landing page for the protected admin workspace.

How it fits into the app: Rendered at `/admin` after `RequireAdmin` and inside `AdminLayout`.

Interview-ready talking points:

Acts as navigation hub for admin problem management features.

Code

```
 1: import { Link } from "react-router-dom";
 2:
 3: export default function AdminDashboardPage() {
 4:   return (
 5:     <section className="px-6 py-8 sm:px-8">
 6:       <div className="rounded-3xl border border-white/15 bg-black/25 p-6 shadow-
[0_18px_60px_rgba(0,0,0,0.28)] backdrop-blur-md">
 7:         <p className="text-xs font-semibold uppercase tracking-[0.28em] text-[#9fb3c8]">
 8:           Dashboard
 9:         </p>
10:         <h1 className="mt-3 text-3xl font-semibold text-white sm:text-4xl">
11:           Manage the platform from a separate admin workspace.
12:         </h1>
13:         <p className="mt-3 max-w-3xl text-sm leading-6 text-[#9eacba] sm:text-base">
14:           Problem creation and moderation live here, separate from the public user-facing routes.
15:         </p>
16:
17:         <div className="mt-6 grid gap-4 md:grid-cols-2">
18:           <Link
19:             to="/admin/problems"
20:             className="rounded-2xl border border-white/12 bg-black/20 p-5 transition hover:bg-black/25"
21:           >
22:             <p className="text-sm font-medium text-white">Manage Problems</p>
23:             <p className="mt-2 text-sm text-[#96a6b5]">
24:               Review the current problem bank, open questions, and remove outdated items.
25:             </p>
26:           </Link>
27:
28:           <Link
29:             to="/admin/problems/new"
30:             className="rounded-2xl border border-white/12 bg-black/20 p-5 transition hover:bg-black/25"
31:           >
32:             <p className="text-sm font-medium text-white">Create New Problem</p>
33:             <p className="mt-2 text-sm text-[#96a6b5]">
34:               Add a new problem statement, examples, constraints, and hidden test cases.
35:             </p>
36:           </Link>
37:         </div>
38:       </div>
39:     </section>
40:   );
41: }
```

Lines 3-17: Defines admin dashboard page.

Lines 18-41: Renders admin action cards linking to problem management and creation.

Admin Problems Page

File: Frontend/src/Pages/Admin/AdminProblemsPage.jsx

Purpose: Lets admins view the problem bank and delete problems.

How it fits into the app: Rendered at `/admin/problems` inside protected admin workspace.

Interview-ready talking points:

Fetches problems from backend and displays admin controls.

Delete action calls backend admin delete endpoint and updates local UI state.

Implementation notes:

Uses browser `window.confirm` for delete confirmation.

Code

```
  1: import { useEffect, useState } from "react";
  2: import { Link } from "react-router-dom";
  3: import axios from "axios";
  4: import { BACKEND_URL } from "../../config/urls.js";
  5:
  6: export default function AdminProblemsPage() {
  7:   const [problems, setProblems] = useState([]);
  8:   const [loading, setLoading] = useState(true);
  9:
 10:   const fetchProblems = async () => {
 11:     try {
 12:       const res = await axios.get(`${BACKEND_URL}/api/problems`, {
 13:         withCredentials: true,
 14:       });
 15:       setProblems(res.data.problems || []);
 16:     } catch {
 17:       setProblems([]);
 18:     } finally {
 19:       setLoading(false);
 20:     }
 21:   };
 22:
 23:   useEffect(() => {
 24:     fetchProblems();
 25:   }, []);
 26:
 27:   const handleDelete = async (id) => {
 28:     if (!window.confirm("Delete this problem?")) return;
 29:
 30:     try {
 31:       await axios.delete(`${BACKEND_URL}/api/problem/${id}`, {
 32:         withCredentials: true,
 33:       });
 34:       setProblems((current) => current.filter((problem) => problem._id !== id));
 35:     } catch (err) {
 36:       alert(err?.response?.data?.message || "Delete failed");
 37:     }
 38:   };
 39:
 40:   return (
 41:     <section className="px-6 py-8 sm:px-8">
 42:       <div className="rounded-3xl border border-white/15 bg-black/25 shadow-
[0_18px_60px_rgba(0,0,0,0.28)] backdrop-blur-md">
 43:         <div className="flex flex-col gap-4 border-b border-white/10 px-6 py-5 sm:flex-row sm:items-
center sm:justify-between">
 44:           <div>
 45:             <p className="text-xs font-semibold uppercase tracking-[0.24em] text-[#8ea1b3]">
 46:               Problem Management
 47:             </p>
 48:             <h1 className="mt-2 text-2xl font-semibold text-white">Review and update the problem
bank</h1>
 49:           </div>
 50:
 51:           <Link
 52:             to="/admin/problems/new"
```

```
 53:             className="rounded-full bg-white px-5 py-3 text-sm font-medium text-[#10212d] transition
hover:bg-[#e7eef5]"
 54:           >
 55:             Add Problem
 56:           </Link>
 57:         </div>
 58:
 59:         {loading && <div className="px-6 py-10 text-sm text-[#9eacba]">Loading problems...</div>}
 60:
 61:         {!loading && !problems.length && (
 62:           <div className="px-6 py-10 text-sm text-[#9eacba]">No problems found.</div>
 63:         )}
 64:
 65:         {!loading && !!problems.length && (
 66:           <div className="overflow-x-auto">
 67:             <table className="min-w-full text-sm">
 68:               <thead className="bg-black/25 text-left text-xs uppercase tracking-[0.2em] text-white/55">
 69:                 <tr>
 70:                   <th className="px-6 py-4">#</th>
 71:                   <th className="px-6 py-4">Title</th>
 72:                   <th className="px-6 py-4 text-right">Acceptance</th>
 73:                   <th className="px-6 py-4 text-right">Actions</th>
 74:                 </tr>
 75:               </thead>
 76:               <tbody className="divide-y divide-white/6">
 77:                 {problems.map((problem) => (
 78:                   <tr key={problem._id} className="hover:bg-white/[0.05]">
 79:                     <td className="px-6 py-4 font-mono text-[#b7c2cc]">
 80:                       {String(problem.problemNumber).padStart(3, "0")}
 81:                     </td>
 82:                     <td className="px-6 py-4 font-medium text-white">{problem.title}</td>
 83:                     <td className="px-6 py-4 text-right text-[#dde6ee]">
 84:                       {(problem.acceptanceRate ?? 0).toFixed(1)}%
 85:                     </td>
 86:                     <td className="px-6 py-4">
 87:                       <div className="flex justify-end gap-2">
 88:                         <Link
 89:                           to={`/problems/${problem._id}`}
 90:                           className="rounded-full border border-white/15 bg-black/20 px-4 py-2 text-xs
font-medium text-white transition hover:bg-black/25"
 91:                         >
 92:                           View
 93:                         </Link>
 94:                         <button
 95:                           onClick={() => handleDelete(problem._id)}
 96:                           className="rounded-full border border-rose-500/30 bg-rose-500/10 px-4 py-2
text-xs font-medium text-rose-200 transition hover:bg-rose-500/15"
 97:                         >
 98:                           Delete
 99:                         </button>
100:                       </div>
101:                     </td>
102:                   </tr>
103:                 ))}
104:               </tbody>
105:             </table>
106:           </div>
107:         )}
108:       </div>
109:     </section>
110:   );
111: }
```

Line 6: Defines admin problem-management page.

Lines 7-9: Stores fetched problems.

Lines 10-22: Defines problem fetch helper.

Lines 23-26: Fetches problems on mount.

```
Lines 27-39: Deletes a problem after confirmation.
Lines 40-111: Renders management table UI.
```

Admin Add Problem Page

File: Frontend/src/Pages/Admin/AdminAddProblemPage.jsx

Purpose: Implements the full admin workflow for creating a problem, visible examples, and hidden test cases.

How it fits into the app: Rendered at `/admin/problems/new`; first posts problem to backend, then posts hidden test cases to a second endpoint.

Interview-ready talking points:

This page shows how the frontend coordinates a two-step admin workflow with dynamic example/test-case rows.

Visible examples and hidden test cases are kept in separate local arrays.

Implementation notes:

Because problem creation and test-case creation are separate API calls, a failure in the second call can leave a partially created problem.

The page uses imperative `alert` messaging for success and failure feedback.

Code

```
import { useState } from "react";
import axios from "axios";
import { useNavigate } from "react-router-dom";
import { BACKEND_URL } from "../../config/urls.js";

function AdminAddProblemPage() {
  const [form, setForm] = useState({
    problemNumber: "",
    title: "",
    statement: "",
    constraints: "",
  });

  const [examples, setExamples] = useState([{ input: "", output: "" }]);
  const [testCases, setTestCases] = useState([{ input: "", output: "" }]);
  const [submitting, setSubmitting] = useState(false);
  const navigate = useNavigate();

  const handleChange = (e) =>
    setForm((current) => ({ ...current, [e.target.name]: e.target.value }));

  const updateArray =
    (setter) =>
    (index, field, value) =>
      setter((rows) =>
        rows.map((row, rowIndex) =>
          rowIndex === index ? { ...row, [field]: value } : row
        )
      );

  const addRow = (setter) => () =>
    setter((rows) => [...rows, { input: "", output: "" }]);

  const rowsValid = (rows) =>
    rows.every((row) => row.input.trim() && row.output.trim());

  const formValid =
    form.problemNumber.trim() &&
    form.title.trim() &&
    form.statement.trim() &&
    form.constraints.trim() &&
    rowsValid(examples) &&
    rowsValid(testCases);

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!formValid) {
      alert("Please fill every field completely.");
      return;
```

```
    }

    try {
      setSubmitting(true);

      const problemResponse = await axios.post(
        `${BACKEND_URL}/api/problem`,
        { ...form, examples },
        { withCredentials: true }
      );
      const problemId = problemResponse.data.problem._id;

      await axios.post(
        `${BACKEND_URL}/api/testcases/${problemId}`,
        { testCases },
        { withCredentials: true }
      );

      alert("Problem and test-cases added successfully");
      navigate("/admin/problems");
    } catch (err) {
      const message = err?.response?.data?.message || "Add failed";
      if (message.includes("E11000")) {
        alert("Problem number already exists.");
      } else {
        console.error(err);
        alert(message);
      }
    } finally {
      setSubmitting(false);
    }
  };

  return (
    <section className="px-6 py-8 sm:px-8">
      <div className="rounded-3xl border border-white/15 bg-black/25 shadow-[0_18px_60px_rgba(0,0,0,0.28)] backdrop-blur-md">
        <div className="border-b border-white/10 px-6 py-5">
          <p className="text-xs font-semibold uppercase tracking-[0.24em] text-[#8ea1b3]">
            Problem Creation
          </p>
          <h1 className="mt-2 text-2xl font-semibold text-white">Create a new coding problem</h1>
          <p className="mt-2 max-w-3xl text-sm leading-6 text-[#95a5b4]">
            Add the public statement, visible examples, and hidden judge test cases from the
            separate admin workspace.
          </p>
        </div>

        <form onSubmit={handleSubmit} className="space-y-6 px-6 py-6">
          <div class="grid gap-4 md:grid-cols-2">
            <label className="block">
              <span className="mb-2 block text-sm font-medium text-[#cad5de]">Problem Number</span>
              <input
                name="problemNumber"
                placeholder="e.g. 101"
                value={form.problemNumber}
                onChange={handleChange}
                className="w-full rounded-2xl border border-white/15 bg-black/25 px-4 py-3 text-white outline-none placeholder:text-white/35"
                required
              />
            </label>

            <label className="block">
              <span className="mb-2 block text-sm font-medium text-[#cad5de]">Title</span>
              <input
                name="title"
                placeholder="Problem title"
                value={form.title}
                onChange={handleChange}
                className="w-full rounded-2xl border border-white/15 bg-black/25 px-4 py-3 text-white
```

```
outline-none placeholder:text-white/35"
                required
              />
            </label>
          </div>

          <label className="block">
            <span className="mb-2 block text-sm font-medium text-[#cad5de]">Statement</span>
            <textarea
              name="statement"
              placeholder="Write the full problem statement"
              value={form.statement}
              onChange={handleChange}
              rows={8}
              className="w-full rounded-2xl border border-white/15 bg-black/25 px-4 py-3 text-white
outline-none placeholder:text-white/35"
              required
            />
          </label>

          <label className="block">
            <span className="mb-2 block text-sm font-medium text-[#cad5de]">Constraints</span>
            <textarea
              name="constraints"
              placeholder="List input bounds and important guarantees"
              value={form.constraints}
              onChange={handleChange}
              rows={4}
              className="w-full rounded-2xl border border-white/15 bg-black/25 px-4 py-3 text-white
outline-none placeholder:text-white/35"
              required
            />
          </label>

          <Section
            title="Examples"
            description="Visible samples shown on the public problem page."
            rows={examples}
            onAdd={addRow(setExamples)}
            onChange={updateArray(setExamples)}
          />

          <Section
            title="Hidden Test Cases"
            description="Private judge cases used to evaluate submissions."
            rows={testCases}
            onAdd={addRow(setTestCases)}
            onChange={updateArray(setTestCases)}
          />

          <div className="flex flex-col gap-3 border-t border-white/10 pt-6 sm:flex-row sm:justify-end">
            <button
              type="button"
              onClick={() => navigate("/admin/problems")}
              className="rounded-2xl border border-white/15 bg-black/20 px-5 py-3 text-sm font-medium
text-white transition hover:bg-black/25"
            >
              Cancel
            </button>
            <button
              disabled={submitting}
              className="rounded-2xl bg-white px-5 py-3 text-sm font-medium text-[#10212d] transition
hover:bg-[#e7eef5] disabled:cursor-not-allowed disabled:opacity-60"
            >
              {submitting ? "Submitting..." : "Create Problem"}
            </button>
          </div>
        </form>
      </div>
    </section>
  );
```

```
185: }
186:
187: const Section = ({ title, description, rows, onAdd, onChange }) => (
188:   <div className="rounded-3xl border border-white/12 bg-black/20 p-5 backdrop-blur-sm">
189:     <h2 className="text-lg font-semibold text-white">{title}</h2>
190:     <p className="mt-1 text-sm text-[#8fa1b3]">{description}</p>
191:     {rows.map((row, index) => (
192:       <div key={index} className="mt-4 grid gap-3 md:grid-cols-2">
193:         <input
194:           placeholder="Input"
195:           value={row.input}
196:           onChange={(e) => onChange(index, "input", e.target.value)}
197:           className="rounded-2xl border border-white/15 bg-black/25 px-4 py-3 text-white outline-none
placeholder:text-white/35"
198:         />
199:         <input
200:           placeholder="Output"
201:           value={row.output}
202:           onChange={(e) => onChange(index, "output", e.target.value)}
203:           className="rounded-2xl border border-white/15 bg-black/25 px-4 py-3 text-white outline-none
placeholder:text-white/35"
204:         />
205:       </div>
206:     ))}
207:     <button
208:       type="button"
209:       onClick={onAdd}
210:       className="mt-4 rounded-2xl border border-white/15 bg-black/20 px-4 py-2 text-sm font-medium text-
white transition hover:bg-black/25"
211:     >
212:       Add Row
213:     </button>
214:   </div>
215: );
216:
217: export default AdminAddProblemPage;
```

Line 6: Defines admin problem-creation page.

Lines 7-13: Stores top-level problem fields.

Line 14: Stores visible example rows.

Lines 15-18: Stores hidden test-case rows.

Lines 19-21: Updates top-level form state.

Lines 22-30: Builds reusable row update helper.

Lines 31-36: Builds reusable row append helper.

Lines 37-44: Calculates overall form validity.

Lines 45-54: Runs the two-step submit workflow.

Lines 55-54: Creates the problem first.

Lines 55-82: Creates hidden test cases second.

Lines 83-186: Renders admin form UI.

Lines 187-217: Defines reusable array-section UI block.

Admin Login Page

File: Frontend/src/Pages/Admin/AdminLoginPage.jsx

Purpose: Implements admin login UI and submits admin-role credentials to the shared backend auth endpoint.

How it fits into the app: Rendered at `/admin/login` inside `AdminAuthLayout`.

Interview-ready talking points:

Uses same backend login endpoint as user login but sends `role: 'admin'`.

Redirects into protected admin workspace on success.

Code

```
  1: import { useEffect, useState } from "react";
  2: import { Link, useNavigate } from "react-router-dom";
  3: import axios from "axios";
  4: import { BACKEND_URL } from "../../config/urls.js";
  5:
  6: export default function AdminLoginPage() {
  7:   const navigate = useNavigate();
  8:   const [formFields, setFormFields] = useState({ email: "", password: "" });
  9:   const [submit, setSubmit] = useState(false);
 10:   const [formError, setFormError] = useState("");
 11:   const [hasSubmitted, setHasSubmitted] = useState(false);
 12:   const [showPassword, setShowPassword] = useState(false);
 13:
 14:   useEffect(() => {
 15:     if (!submit) return;
 16:     setHasSubmitted(true);
 17:
 18:     const { email, password } = formFields;
 19:     if (!email || !password) {
 20:       setFormError("Please fill all the required fields.");
 21:       setSubmit(false);
 22:       return;
 23:     }
 24:
 25:     (async () => {
 26:       try {
 27:         await axios.post(
 28:           `${BACKEND_URL}/api/auth/login`,
 29:           { ...formFields, role: "admin" },
 30:           { withCredentials: true }
 31:         );
 32:         setFormError("");
 33:         navigate("/admin");
 34:       } catch (err) {
 35:         const apiErrors = err?.response?.data?.errors;
 36:         if (Array.isArray(apiErrors) && apiErrors.length) {
 37:           setFormError(apiErrors[0].msg);
 38:         } else {
 39:           setFormError("Admin login failed.");
 40:         }
 41:       }
 42:
 43:       setSubmit(false);
 44:     })();
 45:   }, [formFields, navigate, submit]);
 46:
 47:   return (
 48:     <main className="flex min-h-screen items-center justify-center px-6 py-10">
 49:       <div className="w-full max-w-md overflow-hidden rounded-[32px] border border-white/10 bg-[#171d24]
shadow-[0_24px_80px_rgba(0,0,0,0.45)]">
 50:         <section className="p-8 sm:p-10">
 51:           <h2 className="text-2xl font-semibold text-white">Admin Login</h2>
 52:           <p className="mt-2 text-sm text-[#95a5b4]">Sign in with an administrator account.</p>
 53:
 54:           <form className="mt-8 space-y-5" onSubmit={(e) => e.preventDefault()}>
 55:             <div>
```

```
 56:               <label htmlFor="email" className="mb-1 block text-sm text-[#cad5de]">Email</label>
 57:               <input
 58:                 id="email"
 59:                 type="email"
 60:                 className="w-full rounded-2xl border border-white/10 bg-[#0f1419] px-4 py-3 text-white
outline-none placeholder:text-[#70808f]"
 61:                 onChange={(e) => setFormFields((p) => ({ ...p, email: e.target.value }))}
 62:               />
 63:             </div>
 64:
 65:             <div>
 66:               <label htmlFor="password" className="mb-1 block text-sm text-[#cad5de]">Password</label>
 67:               <div className="relative">
 68:                 <input
 69:                   id="password"
 70:                   type={showPassword ? "text" : "password"}
 71:                   className="w-full rounded-2xl border border-white/10 bg-[#0f1419] px-4 py-3 pr-16 text-
white outline-none placeholder:text-[#70808f]"
 72:                   onChange={(e) => setFormFields((p) => ({ ...p, password: e.target.value }))}
 73:                 />
 74:                 <button
 75:                   type="button"
 76:                   onClick={() => setShowPassword((current) => !current)}
 77:                   className="absolute right-4 top-1/2 -translate-y-1/2 text-xs font-medium text-[#9fb0bf]
transition hover:text-white"
 78:                 >
 79:                   {showPassword ? "Hide" : "Show"}
 80:                 </button>
 81:               </div>
 82:             </div>
 83:
 84:             {hasSubmitted && formError && <p className="text-sm text-rose-300">{formError}</p>}
 85:
 86:             <button
 87:               type="submit"
 88:               onClick={() => setSubmit(true)}
 89:               className="w-full rounded-2xl bg-[#d9e4ef] px-4 py-3 text-sm font-medium text-[#10212d]
transition hover:bg-white"
 90:             >
 91:               Login as Admin
 92:             </button>
 93:           </form>
 94:
 95:           <div className="mt-6 flex flex-col gap-2 text-sm">
 96:             <Link to="/admin/register" className="text-[#dce7f1] hover:text-white">
 97:               Need an admin account?
 98:             </Link>
 99:             <Link to="/login" className="text-[#8ea1b3] hover:text-[#dce7f1]">
100:               Back to user login
101:             </Link>
102:           </div>
103:         </section>
104:       </div>
105:     </main>
106:   );
107: }
```

Lines 6-7: Defines admin login page.

Lines 8-13: Stores login form state.

Lines 14-26: Runs submit-side effect when submit flag changes.

Lines 27-46: Calls backend login endpoint with admin role.

Lines 47-107: Renders admin login UI.

Admin Register Page

File: Frontend/src/Pages/Admin/AdminRegisterPage.jsx

Purpose: Implements admin registration UI with admin secret and stronger password requirements.

How it fits into the app: Rendered at `/admin/register` inside `AdminAuthLayout`.

Interview-ready talking points:

Uses same backend register endpoint as user register but sends `role: 'admin'` plus `adminSecret`.

Separates admin registration flow from public user registration UI.

Implementation notes:

This page reinforces the backend design where admin accounts require a secret.

Code

```
  1: import { useEffect, useState } from "react";
  2: import { Link, useNavigate } from "react-router-dom";
  3: import axios from "axios";
  4: import { BACKEND_URL } from "../../config/urls.js";
  5:
  6: export default function AdminRegisterPage() {
  7:   const navigate = useNavigate();
  8:   const [formFields, setFormFields] = useState({
  9:     userName: "",
 10:     email: "",
 11:     password: "",
 12:     adminSecret: "",
 13:   });
 14:   const [submit, setSubmit] = useState(false);
 15:   const [formError, setFormError] = useState("");
 16:   const [hasSubmitted, setHasSubmitted] = useState(false);
 17:   const [showPassword, setShowPassword] = useState(false);
 18:   const [showAdminSecret, setShowAdminSecret] = useState(false);
 19:
 20:   const passwordRequirements =
 21:     "Use 8-32 characters with at least one uppercase letter, one lowercase letter, one number, and one
special character.";
 22:
 23:   const isStrongPassword = (password) =>
 24:     /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^A-Za-z0-9]).{8,32}$/.test(password);
 25:
 26:   useEffect(() => {
 27:     if (!submit) return;
 28:     setHasSubmitted(true);
 29:
 30:     const { userName, email, password, adminSecret } = formFields;
 31:     if (!userName || !email || !password || !adminSecret) {
 32:       setFormError("Please fill all the required fields.");
 33:       setSubmit(false);
 34:       return;
 35:     }
 36:
 37:     if (!isStrongPassword(password)) {
 38:       setFormError(passwordRequirements);
 39:       setSubmit(false);
 40:       return;
 41:     }
 42:
 43:     (async () => {
 44:       try {
 45:         await axios.post(
 46:           `${BACKEND_URL}/api/auth/register`,
 47:           { ...formFields, role: "admin" },
 48:           { withCredentials: true }
 49:         );
 50:         setFormError("");
 51:         navigate("/admin/login");
 52:       } catch (err) {
 53:         const apiErrors = err?.response?.data?.errors;
 54:         if (Array.isArray(apiErrors) && apiErrors.length) {
```

```
 55:           setFormError(apiErrors[0].msg);
 56:         } else {
 57:           setFormError(err?.response?.data?.message || "Admin registration failed.");
 58:         }
 59:       }
 60:
 61:       setSubmit(false);
 62:     })();
 63:   }, [formFields, navigate, submit]);
 64:
 65:   return (
 66:     <main className="flex min-h-screen items-center justify-center px-6 py-10">
 67:       <div className="grid w-full max-w-5xl overflow-hidden rounded-[32px] border border-white/10 bg-
[#11161c] shadow-[0_24px_80px_rgba(0,0,0,0.45)] md:grid-cols-[1.1fr_0.9fr]">
 68:         <section className="hidden border-r border-white/8 bg-[#0f1419] p-10 md:block">
 69:           <p className="text-xs font-semibold uppercase tracking-[0.3em] text-[#8ea1b3]">
 70:             Admin Registration
 71:           </p>
 72:           <h1 className="mt-4 text-4xl font-semibold leading-tight text-white">
 73:             Create an administrator account in a separate flow.
 74:           </h1>
 75:           <p className="mt-4 max-w-md text-sm leading-7 text-[#95a5b4]">
 76:             Admin registration requires the administrator secret and should stay isolated from the public
sign-up route.
 77:           </p>
 78:         </section>
 79:
 80:         <section className="bg-[#171d24] p-8 sm:p-10">
 81:           <h2 className="text-2xl font-semibold text-white">Admin Registration</h2>
 82:           <p className="mt-2 text-sm text-[#95a5b4]">Create an administrator account.</p>
 83:
 84:           <form className="mt-8 space-y-5" onSubmit={(e) => e.preventDefault()}>
 85:             <div>
 86:               <label htmlFor="userName" className="mb-1 block text-sm text-[#cad5de]">User Name</label>
 87:               <input
 88:                 id="userName"
 89:                 type="text"
 90:                 className="w-full rounded-2xl border border-white/10 bg-[#0f1419] px-4 py-3 text-white
outline-none placeholder:text-[#70808f]"
 91:                 onChange={(e) => setFormFields((p) => ({ ...p, userName: e.target.value }))}
 92:               />
 93:             </div>
 94:
 95:             <div>
 96:               <label htmlFor="email" className="mb-1 block text-sm text-[#cad5de]">Email</label>
 97:               <input
 98:                 id="email"
 99:                 type="email"
100:                 className="w-full rounded-2xl border border-white/10 bg-[#0f1419] px-4 py-3 text-white
outline-none placeholder:text-[#70808f]"
101:                 onChange={(e) => setFormFields((p) => ({ ...p, email: e.target.value }))}
102:               />
103:             </div>
104:
105:             <div>
106:               <label htmlFor="password" className="mb-1 block text-sm text-[#cad5de]">Password</label>
107:               <div className="relative">
108:                 <input
109:                   id="password"
110:                   type={showPassword ? "text" : "password"}
111:                   className="w-full rounded-2xl border border-white/10 bg-[#0f1419] px-4 py-3 pr-16 text-
white outline-none placeholder:text-[#70808f]"
112:                   onChange={(e) => setFormFields((p) => ({ ...p, password: e.target.value }))}
113:                 />
114:                 <button
115:                   type="button"
116:                   onClick={() => setShowPassword((current) => !current)}
117:                   className="absolute right-4 top-1/2 -translate-y-1/2 text-xs font-medium text-[#9fb0bf]
transition hover:text-white"
118:                 >
119:                   {showPassword ? "Hide" : "Show"}
```

```
120:                </button>
121:              </div>
122:              <p className="mt-1 text-xs text-[#7d8f9f]">{passwordRequirements}</p>
123:            </div>
124:
125:            <div>
126:              <label htmlFor="adminSecret" className="mb-1 block text-sm text-[#cad5de]">Admin
Secret</label>
127:              <div className="relative">
128:                <input
129:                  id="adminSecret"
130:                  type={showAdminSecret ? "text" : "password"}
131:                  className="w-full rounded-2xl border border-white/10 bg-[#0f1419] px-4 py-3 pr-16 text-
white outline-none placeholder:text-[#70808f]"
132:                  onChange={(e) => setFormFields((p) => ({ ...p, adminSecret: e.target.value }))}
133:                />
134:                <button
135:                  type="button"
136:                  onClick={() => setShowAdminSecret((current) => !current)}
137:                  className="absolute right-4 top-1/2 -translate-y-1/2 text-xs font-medium text-[#9fb0bf]
transition hover:text-white"
138:                >
139:                  {showAdminSecret ? "Hide" : "Show"}
140:                </button>
141:              </div>
142:            </div>
143:
144:            {hasSubmitted && formError && <p className="text-sm text-rose-300">{formError}</p>}
145:
146:            <button
147:              type="submit"
148:              onClick={() => setSubmit(true)}
149:              className="w-full rounded-2xl bg-[#d9e4ef] px-4 py-3 text-sm font-medium text-[#10212d]
transition hover:bg-white"
150:            >
151:              Create Admin Account
152:            </button>
153:          </form>
154:
155:          <div className="mt-6 flex flex-col gap-2 text-sm">
156:            <Link to="/admin/login" className="text-[#dce7f1] hover:text-white">
157:              Already have an admin account?
158:            </Link>
159:            <Link to="/register" className="text-[#8ea1b3] hover:text-[#dce7f1]">
160:              Back to user registration
161:            </Link>
162:          </div>
163:        </section>
164:      </div>
165:    </main>
166:  );
167: }
```

Lines 6-7: Defines admin register page.

Lines 8-19: Stores admin registration form state.

Lines 20-22: Defines visible password rules.

Lines 23-25: Defines client-side password rule.

Lines 26-44: Runs submit-side effect when submit flag changes.

Lines 45-64: Calls backend register endpoint with admin payload.

Lines 65-167: Renders admin registration UI.

Home Page

File: Frontend/src/Pages/HomePage.jsx

Purpose: Landing page that sets the visual tone and funnels users into main app areas.

How it fits into the app: Rendered at `/` inside `RootLayout`.

Interview-ready talking points:

Uses Framer Motion for gentle hero animation and staged reveals.

Reads Redux auth state to switch CTA between profile and registration.

Implementation notes:

Design uses the mesh background plus overlay gradients as the main visual identity.

Code

```
  1: import { Link } from "react-router-dom";
  2: import { motion } from "framer-motion";
  3: import { useSelector } from "react-redux";
  4: import mesh from "../assets/mesh.png";
  5:
  6: const links = [
  7:   { title: "Problems", to: "/problems" },
  8:   { title: "Explore", to: "/explore" },
  9:   { title: "Contests", to: "/contests" },
 10:   { title: "Leaderboard", to: "/leaderboard" },
 11: ];
 12:
 13: export default function HomePage() {
 14:   const { user } = useSelector((state) => state.auth);
 15:
 16:   return (
 17:     <main className="relative min-h-screen overflow-hidden bg-[#0a0f14] text-white">
 18:       <motion.img
 19:         src={mesh}
 20:         alt="Code Bharat background"
 21:         initial={{ scale: 1.04, opacity: 0.78 }}
 22:         animate={{
 23:           scale: [1.04, 1.08, 1.04],
 24:           opacity: [0.76, 0.9, 0.76],
 25:         }}
 26:         transition={{ duration: 18, repeat: Infinity, ease: "easeInOut" }}
 27:         className="absolute inset-0 h-full w-full object-cover object-center"
 28:       />
 29:
 30:       <div className="absolute inset-0 bg-[linear-
gradient(90deg,rgba(6,10,14,0.78)_0%,rgba(6,10,14,0.48)_45%,rgba(6,10,14,0.3)_100%)]" />
 31:       <div className="absolute inset-0 bg-[radial-
gradient(circle_at_top_right,rgba(255,255,255,0.12),transparent_30%),radial-
gradient(circle_at_bottom_left,rgba(255,255,255,0.08),transparent_28%)]" />
 32:
 33:       <section className="relative mx-auto flex min-h-screen max-w-7xl items-center px-4 py-10 sm:px-6
lg:px-8">
 34:         <div className="max-w-3xl">
 35:           <motion.p
 36:             initial={{ opacity: 0, y: 20 }}
 37:             animate={{ opacity: 1, y: 0 }}
 38:             transition={{ duration: 0.5 }}
 39:             className="text-xs font-semibold uppercase tracking-[0.34em] text-white/70"
 40:           >
 41:             Code Bharat
 42:           </motion.p>
 43:
 44:           <motion.h1
 45:             initial={{ opacity: 0, y: 24 }}
 46:             animate={{ opacity: 1, y: 0 }}
 47:             transition={{ duration: 0.65, delay: 0.06 }}
 48:             className="mt-4 text-4xl font-semibold leading-tight sm:text-5xl lg:text-6xl"
 49:           >
 50:             Practice coding in one serious workspace.
 51:           </motion.h1>
```

```
 52:
 53:           <motion.p
 54:             initial={{ opacity: 0, y: 24 }}
 55:             animate={{ opacity: 1, y: 0 }}
 56:             transition={{ duration: 0.65, delay: 0.14 }}
 57:             className="mt-5 max-w-xl text-base leading-7 text-white/80 sm:text-lg"
 58:           >
 59:             Read. Code. Run. Submit.
 60:           </motion.p>
 61:
 62:           <motion.div
 63:             initial={{ opacity: 0, y: 24 }}
 64:             animate={{ opacity: 1, y: 0 }}
 65:             transition={{ duration: 0.65, delay: 0.22 }}
 66:             className="mt-8 flex flex-wrap gap-3"
 67:           >
 68:             <Link
 69:               to="/problems"
 70:               className="rounded-full bg-white px-6 py-3 text-sm font-medium text-[#10212d] transition
hover:bg-[#e7eef5]"
 71:             >
 72:               Start Solving
 73:             </Link>
 74:
 75:             {user ? (
 76:               <Link
 77:                 to="/profile"
 78:                 className="rounded-full border border-white/20 bg-white/10 px-6 py-3 text-sm font-medium
text-white backdrop-blur transition hover:bg-white/15"
 79:               >
 80:                 {user.userName}
 81:               </Link>
 82:             ) : (
 83:               <Link
 84:                 to="/register"
 85:                 className="rounded-full border border-white/20 bg-white/10 px-6 py-3 text-sm font-medium
text-white backdrop-blur transition hover:bg-white/15"
 86:               >
 87:                 Create Account
 88:               </Link>
 89:             )}
 90:           </motion.div>
 91:
 92:           <motion.div
 93:             initial={{ opacity: 0, y: 28 }}
 94:             animate={{ opacity: 1, y: 0 }}
 95:             transition={{ duration: 0.7, delay: 0.3 }}
 96:             className="mt-12 grid gap-4 sm:grid-cols-2 xl:grid-cols-4"
 97:           >
 98:             {links.map((item) => (
 99:               <Link
100:                 key={item.title}
101:                 to={item.to}
102:                 className="rounded-[24px] border border-white/15 bg-black/20 px-5 py-5 backdrop-blur-md
transition hover:bg-black/28"
103:               >
104:                 <p className="text-lg font-semibold text-white">{item.title}</p>
105:               </Link>
106:             ))}
107:           </motion.div>
108:         </div>
109:       </section>
110:     </main>
111:   );
112: }
```

Lines 6-12: Defines hero quick-link cards.

Line 13: Defines the landing page component.

```
Lines 14-17: Reads auth state to personalize CTA.
Lines 18-43: Animates the mesh background.
Lines 44-74: Renders the animated hero headline.
Lines 75-97: Switches CTA based on login state.
Lines 98-112: Renders landing-page navigation cards.
```

Explore Page

File: Frontend/src/Pages/ExplorePage.jsx

Purpose: Searchable discovery page for browsing problems by title, number, acceptance band, and sorting preference.

How it fits into the app: Rendered at `/explore` inside `RootLayout` with `MeshPageShell`.

Interview-ready talking points:

Fetches the problem list from backend on mount.

Uses `useMemo` for filtered and sorted derived data.

Design emphasizes discovery through stats cards, filter pills, and spotlight links.

Code

```
 1: import { useEffect, useMemo, useState } from "react";
 2: import { Link } from "react-router-dom";
 3: import axios from "axios";
 4: import { BACKEND_URL } from "../config/urls.js";
 5: import MeshPageShell from "../Components/MeshPageShell.jsx";
 6:
 7: const acceptanceFilters = [
 8:   { key: "all", label: "All" },
 9:   { key: "warmup", label: "Warm-Up" },
10:   { key: "balanced", label: "Balanced" },
11:   { key: "challenge", label: "Challenge" },
12: ];
13:
14: export default function ExplorePage() {
15:   const [problems, setProblems] = useState([]);
16:   const [loading, setLoading] = useState(true);
17:   const [search, setSearch] = useState("");
18:   const [filter, setFilter] = useState("all");
19:   const [sortBy, setSortBy] = useState("number");
20:
21:   useEffect(() => {
22:     (async () => {
23:       try {
24:         const res = await axios.get(`${BACKEND_URL}/api/problems`, {
25:           withCredentials: true,
26:         });
27:         setProblems(res.data.problems || []);
28:       } catch {
29:         setProblems([]);
30:       } finally {
31:         setLoading(false);
32:       }
33:     })();
34:   }, []);
35:
36:   const normalizedSearch = search.trim().toLowerCase();
37:
38:   const filteredProblems = useMemo(() => {
39:     return problems
40:       .filter((problem) => {
41:         const matchesSearch =
42:           !normalizedSearch ||
43:           problem.title.toLowerCase().includes(normalizedSearch) ||
44:           String(problem.problemNumber).includes(normalizedSearch);
45:
46:         if (!matchesSearch) return false;
47:
48:         const rate = problem.acceptanceRate ?? 0;
49:         if (filter === "warmup") return rate >= 70;
50:         if (filter === "balanced") return rate >= 40 && rate < 70;
51:         if (filter === "challenge") return rate < 40;
52:         return true;
53:       })
54:       .sort((a, b) => {
55:         if (sortBy === "acceptance-high") return (b.acceptanceRate ?? 0) - (a.acceptanceRate ?? 0);
```

```
 56:         if (sortBy === "acceptance-low") return (a.acceptanceRate ?? 0) - (b.acceptanceRate ?? 0);
 57:         if (sortBy === "title") return a.title.localeCompare(b.title);
 58:         return (a.problemNumber ?? 0) - (b.problemNumber ?? 0);
 59:       });
 60:   }, [filter, normalizedSearch, problems, sortBy]);
 61:
 62:   const averageAcceptance = problems.length
 63:     ? (
 64:         problems.reduce((sum, problem) => sum + (problem.acceptanceRate ?? 0), 0) /
 65:         problems.length
 66:       ).toFixed(1)
 67:     : "0.0";
 68:
 69:   const problemBands = {
 70:     warmup: problems.filter((problem) => (problem.acceptanceRate ?? 0) >= 70).length,
 71:     balanced: problems.filter((problem) => {
 72:       const rate = problem.acceptanceRate ?? 0;
 73:       return rate >= 40 && rate < 70;
 74:     }).length,
 75:     challenge: problems.filter((problem) => (problem.acceptanceRate ?? 0) < 40).length,
 76:   };
 77:
 78:   const spotlight = filteredProblems.slice(0, 5);
 79:
 80:   return (
 81:     <MeshPageShell>
 82:         <div className="grid gap-6 xl:grid-cols-[1.5fr_0.9fr]">
 83:           <section className="rounded-3xl border border-white/15 bg-black/25 p-6 shadow-
[0_18px_60px_rgba(0,0,0,0.28)] backdrop-blur-md">
 84:             <div className="flex flex-col gap-6 border-b border-white/10 pb-6 lg:flex-row lg:items-end
lg:justify-between">
 85:               <div className="max-w-3xl">
 86:                 <p className="text-xs font-semibold uppercase tracking-[0.28em] text-[#9fb3c8]">
 87:                   Practice Library
 88:                 </p>
 89:                 <h1 className="mt-3 text-3xl font-semibold tracking-tight text-white sm:text-4xl">
 90:                   Explore problems by pace, difficulty band, and acceptance trend.
 91:                 </h1>
 92:                 <p className="mt-3 text-sm leading-6 text-[#9eacba] sm:text-base">
 93:                   A cleaner way to move through the bank: search by title or number, sort by
 94:                   acceptance, and pick the right band for your next session.
 95:                 </p>
 96:               </div>
 97:
 98:               <div className="grid grid-cols-3 gap-3 text-left sm:min-w-[360px]">
 99:                 <StatCard label="Live Problems" value={String(problems.length).padStart(2, "0")} />
100:                 <StatCard label="Avg Acceptance" value={`${averageAcceptance}%`} />
101:                 <StatCard label="Challenge Pool" value={`${problemBands.challenge}`} />
102:               </div>
103:             </div>
104:
105:             <div className="mt-6 flex flex-col gap-4">
106:               <div className="grid gap-3 lg:grid-cols-[1fr_200px]">
107:                 <input
108:                   value={search}
109:                   onChange={(e) => setSearch(e.target.value)}
110:                   placeholder="Search by title or problem number"
111:                   className="w-full rounded-2xl border border-white/15 bg-black/25 px-4 py-3 text-sm
text-white outline-none placeholder:text-white/35 focus:border-white/30"
112:                 />
113:
114:                 <select
115:                   value={sortBy}
116:                   onChange={(e) => setSortBy(e.target.value)}
117:                   className="rounded-2xl border border-white/15 bg-black/25 px-4 py-3 text-sm text-white
outline-none focus:border-white/30"
118:                 >
119:                   <option value="number">Sort: Number</option>
120:                   <option value="title">Sort: Title</option>
121:                   <option value="acceptance-high">Sort: Acceptance High-Low</option>
122:                   <option value="acceptance-low">Sort: Acceptance Low-High</option>
```

```
123:                </select>
124:              </div>
125:
126:              <div className="flex flex-wrap gap-2">
127:                {acceptanceFilters.map((item) => (
128:                  <button
129:                    key={item.key}
130:                    type="button"
131:                    onClick={() => setFilter(item.key)}
132:                    className={`rounded-full border px-4 py-2 text-sm transition ${
133:                      filter === item.key
134:                        ? "border-white/25 bg-white text-[#10202f]"
135:                        : "border-white/15 bg-black/25 text-white/85 hover:border-white/25 hover:bg-
black/30"
136:                    }`}
137:                  >
138:                    {item.label}
139:                  </button>
140:                ))}
141:              </div>
142:            </div>
143:          </section>
144:
145:          <aside className="space-y-6">
146:            <Panel title="Problem Bands">
147:              <BandRow label="Warm-Up" count={problemBands.warmup} tone="emerald" />
148:              <BandRow label="Balanced" count={problemBands.balanced} tone="slate" />
149:              <BandRow label="Challenge" count={problemBands.challenge} tone="amber" />
150:            </Panel>
151:          </aside>
152:        </div>
153:
154:        <div className="mt-6 grid gap-6 xl:grid-cols-[1.55fr_0.85fr]">
155:          <section className="overflow-hidden rounded-3xl border border-white/15 bg-black/25 backdrop-
blur-md">
156:            <div className="border-b border-white/10 px-6 py-4">
157:              <div className="flex flex-col gap-2 sm:flex-row sm:items-center sm:justify-between">
158:                <div>
159:                  <h2 className="text-lg font-semibold text-white">Problem Index</h2>
160:                  <p className="text-sm text-[#8ea0b2]">
161:                    {loading ? "Loading problems..." : `${filteredProblems.length} problems in view`}
162:                  </p>
163:                </div>
164:                <p className="text-xs uppercase tracking-[0.22em] text-[#70808f]">
165:                  Acceptance-driven discovery
166:                </p>
167:              </div>
168:            </div>
169:
170:            {loading && <div className="px-6 py-10 text-sm text-[#9eacba]">Loading problems...</div>}
171:
172:            {!loading && !filteredProblems.length && (
173:              <div className="px-6 py-12 text-center text-sm text-[#9eacba]">
174:                No problems matched the current search and filter.
175:              </div>
176:            )}
177:
178:            {!loading && !!filteredProblems.length && (
179:              <div className="overflow-x-auto">
180:                <table className="min-w-full text-sm">
181:                  <thead className="bg-black/25 text-left text-xs uppercase tracking-[0.2em] text-
white/55">
182:                    <tr>
183:                      <th className="px-6 py-4">#</th>
184:                      <th className="px-6 py-4">Title</th>
185:                      <th className="px-6 py-4">Band</th>
186:                      <th className="px-6 py-4 text-right">Acceptance</th>
187:                    </tr>
188:                  </thead>
189:                  <tbody className="divide-y divide-white/6">
190:                    {filteredProblems.map((problem) => {
```

```
                      const rate = problem.acceptanceRate ?? 0;
                      const band =
                        rate >= 70 ? "Warm-Up" : rate >= 40 ? "Balanced" : "Challenge";

                      return (
                        <tr key={problem._id} className="hover:bg-white/[0.05]">
                          <td className="px-6 py-4 font-mono text-[#b3bfca]">
                            {String(problem.problemNumber).padStart(3, "0")}
                          </td>
                          <td className="px-6 py-4">
                            <Link
                              to={`/problems/${problem._id}`}
                              className="font-medium text-white transition hover:text-[#d8e4ef]"
                            >
                              {problem.title}
                            </Link>
                          </td>
                          <td className="px-6 py-4">
                            <span className={`rounded-full px-3 py-1 text-xs font-semibold
${bandClasses(rate)}`}>
                              {band}
                            </span>
                          </td>
                          <td className="px-6 py-4 text-right font-medium text-[#dde6ee]">
                            {rate.toFixed(1)}%
                          </td>
                        </tr>
                      );
                    })}
                  </tbody>
                </table>
              </div>
            )}
          </section>

          <section className="space-y-6">
            <Panel title="Current Spotlight">
              {spotlight.length ? (
                spotlight.map((problem, index) => (
                  <Link
                    key={problem._id}
                    to={`/problems/${problem._id}`}
                    className="flex items-start justify-between gap-4 rounded-2xl border border-white/8
bg-[#12171d] px-4 py-4 transition hover:border-white/15 hover:bg-[#141a21]"
                  >
                    <div>
                      <p className="text-xs uppercase tracking-[0.22em] text-[#6f8090]">
                        Pick {index + 1}
                      </p>
                      <h3 className="mt-1 text-sm font-medium text-white">{problem.title}</h3>
                    </div>
                    <span className="text-sm text-[#c9d4de]">
                      {(problem.acceptanceRate ?? 0).toFixed(1)}%
                    </span>
                  </Link>
                ))
              ) : (
                <p className="text-sm text-[#9eacba]">No spotlight problems available.</p>
              )}
            </Panel>
          </section>
        </div>
      </MeshPageShell>
    );
  }

  function StatCard({ label, value }) {
    return (
      <div className="rounded-2xl border border-white/12 bg-black/20 px-4 py-4 backdrop-blur-sm">
        <p className="text-2xl font-semibold text-white">{value}</p>
        <p className="mt-1 text-xs uppercase tracking-[0.2em] text-[#728394]">{label}</p>
```

```
260:     </div>
261:   );
262: }
263:
264: function Panel({ title, children }) {
265:   return (
266:     <div className="rounded-3xl border border-white/15 bg-black/25 p-5 backdrop-blur-md">
267:       <h2 className="text-sm font-semibold uppercase tracking-[0.24em] text-[#8fa1b3]">{title}</h2>
268:       <div className="mt-4 space-y-3">{children}</div>
269:     </div>
270:   );
271: }
272:
273: function BandRow({ label, count, tone }) {
274:   const palette = {
275:     emerald: "bg-emerald-500/12 text-emerald-200",
276:     slate: "bg-slate-400/12 text-slate-200",
277:     amber: "bg-amber-500/12 text-amber-200",
278:   };
279:
280:   return (
281:     <div className="flex items-center justify-between rounded-2xl border border-white/12 bg-black/20 px-4
py-3 backdrop-blur-sm">
282:       <span className="text-sm text-white">{label}</span>
283:       <span className={`rounded-full px-3 py-1 text-xs font-semibold ${palette[tone]}`}>
284:         {count}
285:       </span>
286:     </div>
287:   );
288: }
289:
290: function bandClasses(rate) {
291:   if (rate >= 70) return "bg-emerald-500/15 text-emerald-200";
292:   if (rate >= 40) return "bg-slate-400/15 text-slate-200";
293:   return "bg-amber-500/15 text-amber-200";
294: }
```

Lines 7-13: Defines the filter pill options.

Line 14: Defines the explore page.

Lines 15-20: Stores fetched problems.

Lines 21-37: Fetches problem data from backend.

Lines 38-61: Computes search/filter/sort result list.

Lines 62-54: Computes summary metrics.

Lines 55-254: Renders the page layout.

Lines 255-263: Defines reusable inline stat card.

Lines 264-272: Defines reusable side panel.

Lines 273-294: Defines band summary row.

Contests Page

File: Frontend/src/Pages/ContestsPage.jsx

Purpose: Builds practice contest packs from the live problem bank instead of using a separate contest API.

How it fits into the app: Rendered at `/contests` inside `RootLayout` with `MeshPageShell`.

Interview-ready talking points:

Fetches problems once and derives different contest sets with `useMemo`.

Demonstrates a frontend-only composition feature built on top of existing backend problem data.

Implementation notes:

Contest packs are heuristic groupings by acceptance rate, not backend-defined contests.

Code

```
  1: import { useEffect, useMemo, useState } from "react";
  2: import { Link } from "react-router-dom";
  3: import axios from "axios";
  4: import { BACKEND_URL } from "../config/urls.js";
  5: import MeshPageShell from "../Components/MeshPageShell.jsx";
  6:
  7: export default function ContestsPage() {
  8:   const [problems, setProblems] = useState([]);
  9:   const [loading, setLoading] = useState(true);
 10:
 11:   useEffect(() => {
 12:     (async () => {
 13:       try {
 14:         const res = await axios.get(`${BACKEND_URL}/api/problems`, {
 15:           withCredentials: true,
 16:         });
 17:         setProblems(res.data.problems || []);
 18:       } catch {
 19:         setProblems([]);
 20:       } finally {
 21:         setLoading(false);
 22:       }
 23:     })();
 24:   }, []);
 25:
 26:   const contestCards = useMemo(() => {
 27:     const easiest = [...problems]
 28:       .sort((a, b) => (b.acceptanceRate ?? 0) - (a.acceptanceRate ?? 0))
 29:       .slice(0, 3);
 30:
 31:     const balanced = [...problems]
 32:       .sort(
 33:         (a, b) =>
 34:           Math.abs((a.acceptanceRate ?? 0) - 55) - Math.abs((b.acceptanceRate ?? 0) - 55)
 35:       )
 36:       .slice(0, 3);
 37:
 38:     const hardest = [...problems]
 39:       .sort((a, b) => (a.acceptanceRate ?? 0) - (b.acceptanceRate ?? 0))
 40:       .slice(0, 4);
 41:
 42:     return [
 43:       {
 44:         title: "Starter Sprint",
 45:         subtitle: "Fast practice round with easier, higher-success problems.",
 46:         duration: "45 min",
 47:         difficulty: "Entry",
 48:         problems: easiest,
 49:       },
 50:       {
 51:         title: "Weekly Balance",
 52:         subtitle: "A measured round designed around mid-band acceptance rates.",
 53:         duration: "75 min",
 54:         difficulty: "Core",
```

```
 55:         problems: balanced,
 56:       },
 57:       {
 58:         title: "Hardcore Marathon",
 59:         subtitle: "Lower-acceptance picks for longer, pressure-heavy sessions.",
 60:         duration: "120 min",
 61:         difficulty: "Advanced",
 62:         problems: hardest,
 63:       },
 64:     ].filter((contest) => contest.problems.length);
 65:   }, [problems]);
 66:
 67:   return (
 68:     <MeshPageShell>
 69:         <section className="rounded-3xl border border-white/15 bg-black/25 p-6 shadow-
[0_18px_60px_rgba(0,0,0,0.28)] backdrop-blur-md">
 70:             <div className="flex flex-col gap-6 border-b border-white/10 pb-6 lg:flex-row lg:items-end
lg:justify-between">
 71:               <div className="max-w-3xl">
 72:                 <p className="text-xs font-semibold uppercase tracking-[0.28em] text-[#a6b6c7]">
 73:                   Practice Contests
 74:                 </p>
 75:                 <h1 className="mt-3 text-3xl font-semibold tracking-tight text-white sm:text-4xl">
 76:                   Structured rounds built directly from your live problem bank.
 77:                 </h1>
 78:                 <p className="mt-3 text-sm leading-6 text-[#98a7b5] sm:text-base">
 79:                   This is a practice-first contest hub. Each pack groups current problems into a
 80:                   timed session so you can train rhythm, pacing, and clean submission habits.
 81:                 </p>
 82:               </div>
 83:
 84:               <div className="grid grid-cols-3 gap-3 text-left sm:min-w-[360px]">
 85:                 <MiniMetric label="Live Problems" value={`${problems.length}`} />
 86:                 <MiniMetric label="Available Rounds" value={`${contestCards.length}`} />
 87:                 <MiniMetric
 88:                   label="Longest Round"
 89:                   value={contestCards.length ? contestCards[contestCards.length - 1].duration : "0"}
 90:                 />
 91:               </div>
 92:             </div>
 93:
 94:             <div className="mt-6 flex flex-wrap gap-3">
 95:               <Link
 96:                 to="/problems"
 97:                 className="rounded-full bg-white px-5 py-3 text-sm font-medium text-[#10212d] transition
hover:bg-[#e7eef5]"
 98:               >
 99:                 Open Problem Bank
100:               </Link>
101:               <Link
102:                 to="/leaderboard"
103:                 className="rounded-full border border-white/15 bg-black/25 px-5 py-3 text-sm font-medium
text-white transition hover:bg-black/30"
104:               >
105:                 View Rankings
106:               </Link>
107:             </div>
108:         </section>
109:
110:         <section className="mt-6 grid gap-6">
111:           {loading && <div className="rounded-3xl border border-white/15 bg-black/25 px-6 py-8 text-sm
text-[#9bacbb] backdrop-blur-md">Loading contest packs...</div>}
112:
113:           {!loading && !contestCards.length && (
114:             <div className="rounded-3xl border border-dashed border-white/15 bg-black/25 px-6 py-12 text-
center text-sm text-[#9bacbb] backdrop-blur-md">
115:               Add a few problems first to generate contest rounds.
116:             </div>
117:           )}
118:
119:           {!loading &&
```

```
120:             contestCards.map((contest) => (
121:               <article
122:                 key={contest.title}
123:                 className="rounded-3xl border border-white/15 bg-black/25 p-6 shadow-
[0_16px_45px_rgba(0,0,0,0.22)] backdrop-blur-md"
124:               >
125:                 <div className="flex flex-col gap-5 border-b border-white/10 pb-5 lg:flex-row lg:items-
start lg:justify-between">
126:                   <div>
127:                     <div className="flex flex-wrap items-center gap-2 text-xs uppercase tracking-[0.22em]
text-[#7c8c9b]">
128:                       <span>{contest.duration}</span>
129:                       <span className="h-1 w-1 rounded-full bg-[#5f7284]" />
130:                       <span>{contest.problems.length} Problems</span>
131:                       <span className="h-1 w-1 rounded-full bg-[#5f7284]" />
132:                       <span>{contest.difficulty}</span>
133:                     </div>
134:                     <h2 className="mt-3 text-2xl font-semibold text-white">{contest.title}</h2>
135:                     <p className="mt-2 max-w-3xl text-sm leading-6 text-[#98a7b5]">
136:                       {contest.subtitle}
137:                     </p>
138:                   </div>
139:
140:                   <Link
141:                     to="/problems"
142:                     className="rounded-full border border-white/15 bg-black/25 px-4 py-2 text-sm font-
medium text-white transition hover:bg-black/30"
143:                   >
144:                     Start Session
145:                   </Link>
146:                 </div>
147:
148:                 <div className="mt-5 overflow-hidden rounded-2xl border border-white/12">
149:                   <table className="min-w-full text-sm">
150:                     <thead className="bg-black/25 text-left text-xs uppercase tracking-[0.2em] text-
white/55">
151:                       <tr>
152:                         <th className="px-5 py-4">Order</th>
153:                         <th className="px-5 py-4">Problem</th>
154:                         <th className="px-5 py-4 text-right">Acceptance</th>
155:                       </tr>
156:                     </thead>
157:                     <tbody className="divide-y divide-white/6">
158:                       {contest.problems.map((problem, index) => (
159:                         <tr key={problem._id} className="hover:bg-white/[0.05]">
160:                           <td className="px-5 py-4 font-mono text-[#aebbc7]">Q{index + 1}</td>
161:                           <td className="px-5 py-4">
162:                             <Link
163:                               to={`/problems/${problem._id}`}
164:                               className="font-medium text-white transition hover:text-[#dde8f2]"
165:                             >
166:                               {problem.title}
167:                             </Link>
168:                           </td>
169:                           <td className="px-5 py-4 text-right text-[#d7e0e8]">
170:                             {(problem.acceptanceRate ?? 0).toFixed(1)}%
171:                           </td>
172:                         </tr>
173:                       ))}
174:                     </tbody>
175:                   </table>
176:                 </div>
177:               </article>
178:             ))}
179:         </section>
180:     </MeshPageShell>
181:   );
182: }
183:
184: function MiniMetric({ label, value }) {
185:   return (
```

```
186:     <div className="rounded-2xl border border-white/12 bg-black/20 px-4 py-4 backdrop-blur-sm">
187:       <p className="text-2xl font-semibold text-white">{value}</p>
188:       <p className="mt-1 text-xs uppercase tracking-[0.2em] text-[#728394]">{label}</p>
189:     </div>
190:   );
191: }
```

Line 7: Defines the contests page.

Lines 8-10: Stores fetched problems.

Lines 11-25: Fetches problem bank from backend.

Lines 26-66: Builds derived contest packs from problem data.

Lines 67-183: Renders contest page layout.

Lines 184-191: Defines inline metric card.

Leaderboard Page

File: Frontend/src/Pages/LeaderboardPage.jsx

Purpose: Shows ranked user standings with podium cards and a full table.

How it fits into the app: Rendered at `/leaderboard` inside `RootLayout` with `MeshPageShell`.

Interview-ready talking points:

Fetches leaderboard data from backend and combines it with current auth user from Redux.

Highlights current user rank if present.

Implementation notes:

Page uses both fetched public data and global auth data together.

Code

```
  1: import { useEffect, useMemo, useState } from "react";
  2: import { useSelector } from "react-redux";
  3: import axios from "axios";
  4: import { BACKEND_URL } from "../config/urls.js";
  5: import MeshPageShell from "../Components/MeshPageShell.jsx";
  6:
  7: export default function LeaderboardPage() {
  8:   const [leaders, setLeaders] = useState([]);
  9:   const [loading, setLoading] = useState(true);
 10:   const { user } = useSelector((state) => state.auth);
 11:
 12:   useEffect(() => {
 13:     (async () => {
 14:       try {
 15:         const res = await axios.get(`${BACKEND_URL}/api/leaderboard`, {
 16:           withCredentials: true,
 17:         });
 18:         setLeaders(res.data.leaderboard || []);
 19:       } catch {
 20:         setLeaders([]);
 21:       } finally {
 22:         setLoading(false);
 23:       }
 24:     })();
 25:   }, []);
 26:
 27:   const podium = leaders.slice(0, 3);
 28:   const currentUserRank = useMemo(
 29:     () => leaders.find((entry) => entry._id === user?._id),
 30:     [leaders, user]
 31:   );
 32:
 33:   return (
 34:     <MeshPageShell>
 35:         <section className="rounded-3xl border border-white/15 bg-black/25 p-6 shadow-
[0_18px_60px_rgba(0,0,0,0.28)] backdrop-blur-md">
 36:             <div className="flex flex-col gap-6 border-b border-white/10 pb-6 lg:flex-row lg:items-end
lg:justify-between">
 37:               <div className="max-w-3xl">
 38:                 <p className="text-xs font-semibold uppercase tracking-[0.28em] text-[#9fb3c8]">
 39:                   Seasonal Ranking
 40:                 </p>
 41:                 <h1 className="mt-3 text-3xl font-semibold tracking-tight text-white sm:text-4xl">
 42:                   A realistic leaderboard built around solved count, accepted runs, and control.
 43:                 </h1>
 44:                 <p className="mt-3 text-sm leading-6 text-[#9eacba] sm:text-base">
 45:                   Rankings prioritize total solved, then accepted submissions, and finally lower
 46:                   submission volume as a tie-breaker for cleaner problem solving.
 47:                 </p>
 48:               </div>
 49:
 50:               <div className="grid grid-cols-3 gap-3 text-left sm:min-w-[360px]">
 51:                 <Metric label="Ranked Coders" value={`${leaders.length}`} />
 52:                 <Metric label="Top Score" value={`${leaders[0]?.solved ?? 0}`} />
 53:                 <Metric label="Your Rank" value={currentUserRank ? `#${currentUserRank.rank}` : "-"} />
```

```
 54:               </div>
 55:             </div>
 56:
 57:             <div className="mt-6 grid gap-4 md:grid-cols-3">
 58:               {podium.map((entry) => (
 59:                 <PodiumCard
 60:                   key={entry._id}
 61:                   entry={entry}
 62:                   highlight={user?._id === entry._id}
 63:                 />
 64:               ))}
 65:             </div>
 66:         </section>
 67:
 68:         <section className="mt-6 overflow-hidden rounded-3xl border border-white/15 bg-black/25 shadow-
[0_16px_45px_rgba(0,0,0,0.22)] backdrop-blur-md">
 69:           <div className="border-b border-white/10 px-6 py-4">
 70:             <div className="flex flex-col gap-2 sm:flex-row sm:items-center sm:justify-between">
 71:               <h2 className="text-lg font-semibold text-white">Full Table</h2>
 72:               <p className="text-xs uppercase tracking-[0.22em] text-[#70808f]">
 73:                 Ranked by solved, accepted, and efficiency
 74:               </p>
 75:             </div>
 76:           </div>
 77:
 78:           {loading && <div className="px-6 py-10 text-sm text-[#9eacba]">Loading leaderboard...</div>}
 79:
 80:           {!loading && !leaders.length && (
 81:             <div className="px-6 py-12 text-center text-sm text-[#9eacba]">
 82:               No leaderboard data is available yet.
 83:             </div>
 84:           )}
 85:
 86:           {!loading && !!leaders.length && (
 87:             <div className="overflow-x-auto">
 88:               <table className="min-w-full text-sm">
 89:                 <thead className="bg-black/25 text-left text-xs uppercase tracking-[0.2em] text-
white/55">
 90:                   <tr>
 91:                     <th className="px-6 py-4">Rank</th>
 92:                     <th className="px-6 py-4">Coder</th>
 93:                     <th className="px-6 py-4 text-right">Solved</th>
 94:                     <th className="px-6 py-4 text-right">Accepted</th>
 95:                     <th className="px-6 py-4 text-right">Submissions</th>
 96:                     <th className="px-6 py-4 text-right">Acceptance</th>
 97:                   </tr>
 98:                 </thead>
 99:                 <tbody className="divide-y divide-white/6">
100:                   {leaders.map((entry) => {
101:                     const isCurrentUser = user?._id === entry._id;
102:
103:                     return (
104:                       <tr
105:                         key={entry._id}
106:                         className={isCurrentUser ? "bg-white/[0.08]" : "hover:bg-white/[0.05]"}
107:                       >
108:                         <td className="px-6 py-4 font-semibold text-white">#{entry.rank}</td>
109:                         <td className="px-6 py-4">
110:                           <div className="flex items-center gap-3">
111:                             <div className="flex h-10 w-10 items-center justify-center rounded-full bg-
white/10 text-sm font-semibold uppercase text-white">
112:                               {entry.userName?.[0] || "U"}
113:                             </div>
114:                             <div>
115:                               <p className="font-medium text-white">
116:                                 {entry.userName}
117:                                 {isCurrentUser ? " (You)" : ""}
118:                               </p>
119:                               <p className="text-xs text-[#7c8e9f]">
120:                                 {entry.institution || "Independent Coder"}
121:                               </p>
```

```
122:                            </div>
123:                          </div>
124:                        </td>
125:                        <td className="px-6 py-4 text-right text-white">{entry.solved}</td>
126:                        <td className="px-6 py-4 text-right text-white">{entry.acceptedSubmissions}</td>
127:                        <td className="px-6 py-4 text-right text-[#cad5de]">{entry.totalSubmissions}</td>
128:                        <td className="px-6 py-4 text-right text-[#d9e3ea]">
129:                          {entry.acceptanceRate.toFixed(1)}%
130:                        </td>
131:                      </tr>
132:                    );
133:                  })}
134:                </tbody>
135:              </table>
136:            </div>
137:          )}
138:        </section>
139:      </MeshPageShell>
140:    );
141: }
142:
143: function Metric({ label, value }) {
144:   return (
145:     <div className="rounded-2xl border border-white/12 bg-black/20 px-4 py-4 backdrop-blur-sm">
146:       <p className="text-2xl font-semibold text-white">{value}</p>
147:       <p className="mt-1 text-xs uppercase tracking-[0.2em] text-[#728394]">{label}</p>
148:     </div>
149:   );
150: }
151:
152: function PodiumCard({ entry, highlight }) {
153:   return (
154:     <div className={`rounded-2xl border px-5 py-5 backdrop-blur-sm ${highlight ? "border-white/25 bg-
white/[0.12]" : "border-white/12 bg-black/20"}`}>
155:       <div className="flex items-center gap-3">
156:         <div className="flex h-11 w-11 items-center justify-center rounded-full bg-white/10 text-sm font-
semibold uppercase text-white">
157:           {entry.userName?.[0] || "U"}
158:         </div>
159:         <div>
160:           <p className="text-sm font-medium text-white">{entry.userName}</p>
161:           <p className="text-xs text-[#7d8c9a]">Rank #{entry.rank}</p>
162:         </div>
163:       </div>
164:
165:       <div className="mt-4 grid grid-cols-3 gap-2">
166:         <PodiumStat label="Solved" value={entry.solved} />
167:         <PodiumStat label="Accepted" value={entry.acceptedSubmissions} />
168:         <PodiumStat label="Rate" value={`${entry.acceptanceRate.toFixed(1)}%`} />
169:       </div>
170:     </div>
171:   );
172: }
173:
174: function PodiumStat({ label, value }) {
175:   return (
176:     <div className="rounded-xl border border-white/12 bg-black/20 px-3 py-3 text-center backdrop-blur-
sm">
177:       <p className="text-sm font-semibold text-white">{value}</p>
178:       <p className="mt-1 text-[11px] uppercase tracking-[0.2em] text-[#6d7f8f]">{label}</p>
179:     </div>
180:   );
181: }
```

Line 7: Defines leaderboard page.

Lines 8-11: Stores fetched leaderboard data.

Lines 12-26: Fetches leaderboard from backend.

Line 27: Computes top three cards.

```
Lines 28-32: Finds current user's rank.
Lines 33-142: Renders leaderboard layout.
Lines 143-151: Defines summary metric card.
Lines 152-173: Defines podium card.
Lines 174-181: Defines small podium stat card.
```

Login Page

File: Frontend/src/Pages/LoginPage.jsx

Purpose: Implements user login form UI and client-side submission flow.

How it fits into the app: Rendered at `/login` inside `LoginLayout`.

Interview-ready talking points:

Uses local form state and a submit trigger flag instead of form libraries.

Sends `{ role: 'user' }` to backend login endpoint.

Navigates to home on successful login.

Implementation notes:

Password visibility toggle is implemented locally in the page.

Code

```
 1: import { useEffect, useState } from "react";
 2: import { useNavigate, Link } from "react-router-dom";
 3: import axios from "axios";
 4: import mesh from "../assets/mesh.png";
 5: import { BACKEND_URL } from "../config/urls.js";
 6:
 7: export default function LoginPage() {
 8:   const navigate = useNavigate();
 9:   const [formFields, setFormFields] = useState({ email: "", password: "" });
10:   const [submit, setSubmit] = useState(false);
11:   const [formError, setFormError] = useState("");
12:   const [hasSubmitted, setHasSubmitted] = useState(false);
13:   const [showPassword, setShowPassword] = useState(false);
14:
15:   useEffect(() => {
16:     if (!submit) return;
17:     setHasSubmitted(true);
18:     const { email, password } = formFields;
19:     if (!email || !password) {
20:       setFormError("Please fill all the required fields.");
21:       setSubmit(false);
22:       return;
23:     }
24:     (async () => {
25:       try {
26:         await axios.post(
27:           `${BACKEND_URL}/api/auth/login`,
28:           { ...formFields, role: "user" },
29:           { withCredentials: true }
30:         );
31:         setFormError("");
32:         navigate("/");
33:       } catch (err) {
34:         console.error("Login error", err.response?.data || err.message);
35:         const apiErrors = err?.response?.data?.errors;
36:         if (Array.isArray(apiErrors) && apiErrors.length) {
37:           setFormError(apiErrors[0].msg);
38:         } else {
39:           setFormError("Login failed.");
40:         }
41:       }
42:       setSubmit(false);
43:     })();
44:   }, [submit, formFields, navigate]);
45:
46:   return (
47:     <main className="relative min-h-screen overflow-hidden bg-[#0a0f14] text-white">
48:       <img
49:         src={mesh}
50:         alt="Code Bharat background"
51:         className="absolute inset-0 h-full w-full object-cover object-center"
52:       />
53:       <div className="absolute inset-0 bg-[linear-
```

```
gradient(90deg,rgba(6,10,14,0.72)_0%,rgba(6,10,14,0.46)_45%,rgba(6,10,14,0.38)_100%)]" />
 54:       <div className="absolute inset-0 bg-[radial-
gradient(circle_at_top_right,rgba(255,255,255,0.12),transparent_32%),radial-
gradient(circle_at_bottom_left,rgba(255,255,255,0.08),transparent_28%)]" />
 55:
 56:       <section className="relative mx-auto flex min-h-screen max-w-7xl items-center justify-end px-4 py-
10 sm:px-6 lg:px-8">
 57:         <div className="w-full max-w-md rounded-[30px] border border-white/15 bg-black/25 p-7 backdrop-
blur-md sm:p-8">
 58:           <p className="text-xs font-semibold uppercase tracking-[0.32em] text-white/70">
 59:             Login
 60:           </p>
 61:           <h1 className="mt-4 text-3xl font-semibold text-white">Welcome back</h1>
 62:           <p className="mt-2 text-sm text-white/75">Continue solving.</p>
 63:
 64:           <form className="mt-8 space-y-5" onSubmit={(e) => e.preventDefault()}>
 65:             <div>
 66:               <label htmlFor="email" className="mb-2 block text-sm font-medium text-white/85">
 67:                 Email
 68:               </label>
 69:               <input
 70:                 id="email"
 71:                 name="email"
 72:                 type="email"
 73:                 className="w-full rounded-2xl border border-white/15 bg-black/25 px-4 py-3 text-white
outline-none placeholder:text-white/35"
 74:                 onChange={(e) => setFormFields((p) => ({ ...p, email: e.target.value }))}
 75:               />
 76:             </div>
 77:
 78:             <div>
 79:               <label htmlFor="password" className="mb-2 block text-sm font-medium text-white/85">
 80:                 Password
 81:               </label>
 82:               <div className="relative">
 83:                 <input
 84:                   id="password"
 85:                   name="password"
 86:                   type={showPassword ? "text" : "password"}
 87:                   className="w-full rounded-2xl border border-white/15 bg-black/25 px-4 py-3 pr-16 text-
white outline-none placeholder:text-white/35"
 88:                   onChange={(e) => setFormFields((p) => ({ ...p, password: e.target.value }))}
 89:                 />
 90:                 <button
 91:                   type="button"
 92:                   onClick={() => setShowPassword((current) => !current)}
 93:                   className="absolute right-4 top-1/2 -translate-y-1/2 text-xs font-medium text-white/70
transition hover:text-white"
 94:                 >
 95:                   {showPassword ? "Hide" : "Show"}
 96:                 </button>
 97:               </div>
 98:             </div>
 99:
100:             {hasSubmitted && formError && <p className="text-sm text-rose-200">{formError}</p>}
101:
102:             <button
103:               type="submit"
104:               onClick={() => setSubmit(true)}
105:               className="w-full rounded-full bg-white px-6 py-3 text-sm font-medium text-[#10212d]
transition hover:bg-[#e7eef5]"
106:             >
107:               Sign In
108:             </button>
109:           </form>
110:
111:           <p className="mt-6 text-sm text-white/72">
112:             New here?{" "}
113:             <Link to="/register" className="font-medium text-white">
114:               Create account
115:             </Link>
```

```
116:           </p>
117:         </div>
118:       </section>
119:     </main>
120:   );
121: }
```

Lines 7-8: Defines login page.

Lines 9-14: Stores form values.

Lines 15-25: Runs submit-side effect when `submit` flag flips.

Lines 26-19: Calls backend login endpoint.

Lines 20-45: Stores API or validation error message.

Lines 46-121: Renders login form UI.

Not Found Page

File: Frontend/src/Pages/NotFoundPage.jsx

Purpose: Simple fallback 404 screen component.

How it fits into the app: Intended for unmatched routes, but not currently wired into the router tree.

Interview-ready talking points:

Good interview point: component exists, but router integration is missing.

Code

```
1: export default function NotFoundPage() {
2:   return (
3:     <div className="min-h-screen bg-[#0f0f0f] text-center flex items-center justify-center text-white">
4:       <h1 className="text-3xl">404 - Page Not Found</h1>
5:     </div>
6:   );
7: }
```

Lines 1-3: Defines the 404 page component.

Lines 4-7: Renders the 404 message.

Problem Set Page

File: Frontend/src/Pages/ProblemSetPage.jsx

Purpose: Displays the full question bank with search, band filters, stats, and direct navigation to each problem.

How it fits into the app: Rendered at `/problems` inside `ProblemLayout`; data comes from a route loader.

Interview-ready talking points:

Uses React Router loader for data fetch instead of `useEffect` inside component.

Computes filtered problem view with `useMemo`.

Implementation notes:

Design mirrors Explore page but is more list-oriented and directly task-focused.

Code

```
  1: import axios from "axios";
  2: import { useMemo, useState } from "react";
  3: import { Link, useLoaderData } from "react-router-dom";
  4: import { BACKEND_URL } from "../config/urls.js";
  5: import MeshPageShell from "../Components/MeshPageShell.jsx";
  6:
  7: function ProblemSetPage() {
  8:   const { problems } = useLoaderData();
  9:   const [search, setSearch] = useState("");
 10:   const [band, setBand] = useState("all");
 11:
 12:   const normalizedSearch = search.trim().toLowerCase();
 13:
 14:   const filteredProblems = useMemo(() => {
 15:     return problems.filter((problem) => {
 16:       const matchesSearch =
 17:         !normalizedSearch ||
 18:         problem.title.toLowerCase().includes(normalizedSearch) ||
 19:         String(problem.problemNumber).includes(normalizedSearch);
 20:
 21:       if (!matchesSearch) return false;
 22:
 23:       const rate = problem.acceptanceRate ?? 0;
 24:       if (band === "warmup") return rate >= 70;
 25:       if (band === "balanced") return rate >= 40 && rate < 70;
 26:       if (band === "challenge") return rate < 40;
 27:       return true;
 28:     });
 29:   }, [band, normalizedSearch, problems]);
 30:
 31:   const stats = {
 32:     total: problems.length,
 33:     warmup: problems.filter((problem) => (problem.acceptanceRate ?? 0) >= 70).length,
 34:     challenge: problems.filter((problem) => (problem.acceptanceRate ?? 0) < 40).length,
 35:     average: problems.length
 36:       ? (
 37:           problems.reduce((sum, problem) => sum + (problem.acceptanceRate ?? 0), 0) /
 38:           problems.length
 39:         ).toFixed(1)
 40:       : "0.0",
 41:   };
 42:
 43:   return (
 44:     <MeshPageShell>
 45:         <section className="rounded-3xl border border-white/15 bg-black/25 p-6 shadow-
[0_18px_60px_rgba(0,0,0,0.28)] backdrop-blur-md">
 46:           <div className="flex flex-col gap-6 border-b border-white/10 pb-6 xl:flex-row xl:items-end
xl:justify-between">
 47:             <div className="max-w-3xl">
 48:               <p className="text-xs font-semibold uppercase tracking-[0.28em] text-[#9fb3c8]">
 49:                 Problem Set
 50:               </p>
 51:               <h1 className="mt-3 text-3xl font-semibold tracking-tight text-white sm:text-4xl">
```

```
 52:                 Browse the full question bank with cleaner filtering and faster scanning.
 53:               </h1>
 54:               <p className="mt-3 text-sm leading-6 text-[#9eacba] sm:text-base">
 55:                 Use search for direct lookup, then narrow by acceptance band when you want an
 56:                 easier warm-up or a harder practice block.
 57:               </p>
 58:             </div>
 59:
 60:             <div className="grid grid-cols-2 gap-3 xl:min-w-[360px]">
 61:               <Metric label="Total" value={`${stats.total}`} />
 62:               <Metric label="Warm-Up" value={`${stats.warmup}`} />
 63:               <Metric label="Challenge" value={`${stats.challenge}`} />
 64:               <Metric label="Avg Rate" value={`${stats.average}%`} />
 65:             </div>
 66:           </div>
 67:
 68:           <div className="mt-6">
 69:             <div className="grid w-full gap-3 lg:grid-cols-[1fr_auto]">
 70:               <input
 71:                 value={search}
 72:                 onChange={(e) => setSearch(e.target.value)}
 73:                 placeholder="Search by title or problem number"
 74:                 className="w-full rounded-2xl border border-white/15 bg-black/25 px-4 py-3 text-sm text-
white outline-none placeholder:text-white/35 focus:border-white/30"
 75:               />
 76:
 77:               <div className="flex flex-wrap gap-2">
 78:                 {[
 79:                   ["all", "All"],
 80:                   ["warmup", "Warm-Up"],
 81:                   ["balanced", "Balanced"],
 82:                   ["challenge", "Challenge"],
 83:                 ].map(([value, label]) => (
 84:                   <button
 85:                     key={value}
 86:                     type="button"
 87:                     onClick={() => setBand(value)}
 88:                     className={`rounded-full border px-4 py-2 text-sm transition ${
 89:                       band === value
 90:                         ? "border-white/25 bg-white text-[#10202f]"
 91:                         : "border-white/15 bg-black/25 text-white/85 hover:border-white/25 hover:bg-
black/30"
 92:                     }`}
 93:                   >
 94:                     {label}
 95:                   </button>
 96:                 ))}
 97:               </div>
 98:             </div>
 99:           </div>
100:         </section>
101:
102:         <section className="mt-6 overflow-hidden rounded-3xl border border-white/15 bg-black/25 shadow-
[0_16px_45px_rgba(0,0,0,0.22)] backdrop-blur-md">
103:           <div className="border-b border-white/10 px-6 py-4">
104:             <div className="flex flex-col gap-2 sm:flex-row sm:items-center sm:justify-between">
105:               <h2 className="text-lg font-semibold text-white">All Problems</h2>
106:               <p className="text-sm text-[#8ea0b2]">{filteredProblems.length} shown</p>
107:             </div>
108:           </div>
109:
110:           {!filteredProblems.length && (
111:             <div className="px-6 py-12 text-center text-sm text-[#9eacba]">
112:               No problems matched the current search and filter.
113:             </div>
114:           )}
115:
116:           {!!filteredProblems.length && (
117:             <div className="overflow-x-auto">
118:               <table className="min-w-full text-sm">
119:                 <thead className="bg-black/25 text-left text-xs uppercase tracking-[0.2em] text-
```

```
white/55">
                  <tr>
                    <th className="px-6 py-4">#</th>
                    <th className="px-6 py-4">Title</th>
                    <th className="px-6 py-4">Band</th>
                    <th className="px-6 py-4 text-right">Acceptance</th>
                  </tr>
                </thead>
                <tbody className="divide-y divide-white/6">
                  {filteredProblems.map((problem) => {
                    const rate = problem.acceptanceRate ?? 0;
                    const badge =
                      rate >= 70
                        ? "bg-emerald-500/15 text-emerald-200"
                        : rate >= 40
                        ? "bg-slate-400/15 text-slate-200"
                        : "bg-amber-500/15 text-amber-200";
                    const label =
                      rate >= 70 ? "Warm-Up" : rate >= 40 ? "Balanced" : "Challenge";

                    return (
                      <tr key={problem._id} className="hover:bg-white/[0.05]">
                        <td className="px-6 py-4 font-mono text-[#b3bfca]">
                          {String(problem.problemNumber).padStart(3, "0")}
                        </td>
                        <td className="px-6 py-4">
                          <Link
                            to={`/problems/${problem._id}`}
                            className="font-medium text-white transition hover:text-[#d8e4ef]"
                          >
                            {problem.title}
                          </Link>
                        </td>
                        <td className="px-6 py-4">
                          <span className={`rounded-full px-3 py-1 text-xs font-semibold ${badge}`}>
                            {label}
                          </span>
                        </td>
                        <td className="px-6 py-4 text-right font-medium text-[#dde6ee]">
                          {rate.toFixed(1)}%
                        </td>
                      </tr>
                    );
                  })}
                </tbody>
              </table>
            </div>
          )}
        </section>
    </MeshPageShell>
  );
}

function Metric({ label, value }) {
  return (
    <div className="min-w-0 rounded-2xl border border-white/12 bg-black/20 px-4 py-4 backdrop-blur-sm">
      <p className="truncate text-2xl font-semibold text-white">{value}</p>
      <p className="mt-1 text-[11px] uppercase tracking-[0.18em] text-[#728394]">{label}</p>
    </div>
  );
}

export default ProblemSetPage;

export const problemSetLoader = async () => {
  const res = await axios.get(`${BACKEND_URL}/api/problems`, {
    withCredentials: true,
  });
  return { problems: res.data.problems };
};
```

```
Line 7: Defines the problem-set page.
Line 8: Receives preloaded data from loader.
Lines 9-13: Stores filter UI state.
Lines 14-30: Computes filtered list.
Lines 31-42: Computes summary metrics.
Lines 43-171: Renders problem-set layout.
Lines 172-182: Defines inline stat card.
Lines 183-188: Defines route loader for problem set data.
```

Problem Workspace Page

File: Frontend/src/Pages/ProblemPage.jsx

Purpose: Core coding workspace page that shows the problem statement, code editor, console, AI review, and submission history.

How it fits into the app: Rendered at `/problems/:problemId` inside `ProblemLayout`; data comes from a route loader and user actions trigger both backend and compiler-service calls.

Interview-ready talking points:

This is the strongest file to discuss in interviews because it combines layout, loaders, local state, editor integration, split panes, markdown rendering, backend calls, compiler calls, and submission history.

Uses `react-split` for resizable panels and Monaco for code editing.

Calls compiler service for run/review and backend for submit/submission history.

Implementation notes:

The language effect contains a Java template, but Java is not exposed in the select options.

Problem loader redirects to `/login` on backend 401.

Code

```
 1: import { useState, useEffect } from "react";
 2: import { useLoaderData, useParams, redirect } from "react-router-dom";
 3: import axios from "axios";
 4: import Split from "react-split";
 5: import Editor from "@monaco-editor/react";
 6: import ReactMarkdown from "react-markdown";
 7: import remarkGfm from "remark-gfm";
 8: import SubmissionTable from "../Components/SubmissionTable.jsx";
 9: import { BACKEND_URL, COMPILER_URL } from "../config/urls.js";
10:
11: function ProblemPage() {
12:   const { problem } = useLoaderData();
13:   const examples = problem.examples ?? [];
14:
15:   const [code, setCode] = useState(
16:     `#include <iostream>\nusing namespace std;\n\nint main() {\n    \n}`
17:   );
18:   const [input, setInput] = useState("");
19:   const [output, setOutput] = useState("");
20:   const [review, setReview] = useState("");
21:   const [language, setLanguage] = useState("cpp");
22:
23:   const [loading, setLoading] = useState(false);
24:   const [reviewLoading, setReviewLoading] = useState(false);
25:   const { problemId } = useParams();
26:
27:   const [subsOpen, setSubsOpen] = useState(false);
28:   const [subs, setSubs] = useState([]);
29:
30:   const acceptanceRate = problem.acceptanceRate ?? 0;
31:   const band = acceptanceRate >= 70 ? "Warm-Up" : acceptanceRate >= 40 ? "Balanced" : "Challenge";
32:
33:   const fetchSubs = async () => {
34:     try {
35:       const { data } = await axios.get(
36:         `${BACKEND_URL}/api/problem/${problemId}/submissions`,
37:         { withCredentials: true }
38:       );
39:       setSubs(data.submissions);
40:     } catch {
41:       setSubs([]);
42:     }
43:   };
44:
45:   const toggleSubs = () => {
46:     const next = !subsOpen;
47:     setSubsOpen(next);
48:     if (next && !subs.length) fetchSubs();
```

```
  49:   };
  50:
  51:   const handleRun = async () => {
  52:     setLoading(true);
  53:     setOutput("");
  54:     try {
  55:       const { data } = await axios.post(`${COMPILER_URL}/run`, {
  56:         code,
  57:         language,
  58:         input,
  59:       });
  60:       setOutput(
  61:         typeof data.output === "object"
  62:           ? JSON.stringify(data.output, null, 2)
  63:           : data.output
  64:       );
  65:     } catch (err) {
  66:       setOutput(err?.response?.data?.error || "Run error");
  67:     } finally {
  68:       setLoading(false);
  69:     }
  70:   };
  71:
  72:   const handleSubmit = async () => {
  73:     setLoading(true);
  74:     setOutput("");
  75:     try {
  76:       const { data } = await axios.post(
  77:         `${BACKEND_URL}/api/problem/${problem._id}/submit`,
  78:         { code, language },
  79:         { withCredentials: true }
  80:       );
  81:       setOutput(`Submission Result: ${data.status}`);
  82:       fetchSubs();
  83:     } catch (err) {
  84:       setOutput(err?.response?.data?.message || "Submission error");
  85:     } finally {
  86:       setLoading(false);
  87:     }
  88:   };
  89:
  90:   const handleReview = async () => {
  91:     if (reviewLoading) return;
  92:     setReview("");
  93:     setReviewLoading(true);
  94:     try {
  95:       const { data } = await axios.post(`${COMPILER_URL}/ai-review`, { code });
  96:       setReview(data.review);
  97:     } catch {
  98:       setReview("AI review failed");
  99:     } finally {
 100:       setReviewLoading(false);
 101:     }
 102:   };
 103:
 104:   useEffect(() => {
 105:     if (language === "cpp")
 106:       setCode(
 107:         `#include <iostream>\nusing namespace std;\n\nint main() {\n    \n}`
 108:       );
 109:     else if (language === "java")
 110:       setCode(
 111:         `public class Main {\n    public static void main(String[] args) {\n        \n    }\n}`
 112:       );
 113:     else if (language === "python") setCode(`print("Hello, world!")`);
 114:     else if (language === "javascript") setCode(`console.log("Hello, world!");`);
 115:     else if (language === "go")
 116:       setCode(
 117:         `package main\nimport "fmt"\nfunc main() {\n    fmt.Println("Hello, world!")\n}`
 118:       );
 119:     else if (language === "c")
```

```
120:       setCode(`#include <stdio.h>\nint main() {\n    return 0;\n}`);
121:   }, [language]);
122:
123:   return (
124:     <main className="min-h-screen bg-[#0f1419] text-[#e8edf2]">
125:       <section className="border-b border-white/10 bg-[#171d24]">
126:         <div className="mx-auto max-w-[1600px] px-4 py-6 sm:px-6 lg:px-8">
127:           <div className="flex flex-col gap-5 xl:flex-row xl:items-start xl:justify-between">
128:             <div className="max-w-4xl">
129:               <div className="flex flex-wrap items-center gap-2 text-xs uppercase tracking-[0.22em] text-
[#8092a3]">
130:                 <span>Problem {String(problem.problemNumber ?? 0).padStart(3, "0")}</span>
131:                 <span className="h-1 w-1 rounded-full bg-[#5f7284]" />
132:                 <span>{band}</span>
133:               </div>
134:               <h1 className="mt-3 text-3xl font-semibold tracking-tight text-white">
135:                 {problem.title}
136:               </h1>
137:               <div className="mt-4 grid gap-3 sm:grid-cols-3">
138:                 <TopStat label="Acceptance" value={`${acceptanceRate.toFixed(1)}%`} />
139:                 <TopStat label="Accepted Runs" value={`${problem.acceptance ?? 0}`} />
140:                 <TopStat label="Total Submissions" value={`${problem.totalSubmissions ?? 0}`} />
141:               </div>
142:             </div>
143:
144:             <div className="flex flex-wrap gap-3">
145:               <button
146:                 onClick={toggleSubs}
147:                 className="rounded-full border border-white/12 bg-[#12171d] px-4 py-2 text-sm font-medium
text-white transition hover:bg-[#141a21]"
148:               >
149:                 {subsOpen ? "Hide Submissions" : "View Submissions"}
150:               </button>
151:             </div>
152:           </div>
153:         </div>
154:       </section>
155:
156:       <section className="mx-auto max-w-[1600px] px-4 py-6 sm:px-6 lg:px-8">
157:         {subsOpen && (
158:           <div className="mb-6 rounded-3xl border border-white/10 bg-[#171d24] p-5">
159:             <div className="mb-4">
160:               <h2 className="text-lg font-semibold text-white">Submission History</h2>
161:               <p className="text-sm text-[#95a5b4]">Your runs on this problem.</p>
162:             </div>
163:             <SubmissionTable submissions={subs} />
164:           </div>
165:         )}
166:
167:         <Split className="flex min-h-[calc(100vh-220px)] gap-0" gutterSize={10} sizes={[42, 58]}>
168:           <article className="overflow-y-auto rounded-3xl border border-white/10 bg-[#171d24]">
169:             <div className="border-b border-white/10 px-6 py-5">
170:               <h2 className="text-lg font-semibold text-white">Statement</h2>
171:             </div>
172:
173:             <div className="space-y-8 px-6 py-6">
174:               <section>
175:                 <pre className="whitespace-pre-wrap text-sm leading-7 text-[#d5dee6]">
176:                   {problem.statement}
177:                 </pre>
178:               </section>
179:
180:               <section className="space-y-4">
181:                 <div className="flex items-center justify-between">
182:                   <h3 className="text-base font-semibold text-white">Examples</h3>
183:                   <span className="text-xs uppercase tracking-[0.2em] text-[#758697]">
184:                     {examples.length} sample{examples.length === 1 ? "" : "s"}
185:                   </span>
186:                 </div>
187:
188:                 {examples.map((example, index) => (
```

```
189:                    <div
190:                      key={index}
191:                      className="rounded-2xl border border-white/8 bg-[#12171d] p-5"
192:                    >
193:                      <div className="grid gap-4 lg:grid-cols-2">
194:                        <div>
195:                          <p className="text-xs uppercase tracking-[0.2em] text-[#718191]">
196:                            Input {index + 1}
197:                          </p>
198:                          <pre className="mt-2 whitespace-pre-wrap text-sm leading-6 text-[#e3ebf2]">
199:                            {example.input}
200:                          </pre>
201:                        </div>
202:                        <div>
203:                          <p className="text-xs uppercase tracking-[0.2em] text-[#718191]">
204:                            Expected Output
205:                          </p>
206:                          <pre className="mt-2 whitespace-pre-wrap text-sm leading-6 text-[#e3ebf2]">
207:                            {example.output}
208:                          </pre>
209:                        </div>
210:                      </div>
211:                    </div>
212:                  ))}
213:                </section>
214:
215:                <section className="rounded-2xl border border-white/8 bg-[#12171d] p-5">
216:                  <h3 className="text-base font-semibold text-white">Constraints</h3>
217:                  <pre className="mt-3 whitespace-pre-wrap text-sm leading-6 text-[#d5dee6]">
218:                    {problem.constraints}
219:                  </pre>
220:                </section>
221:              </div>
222:            </article>
223:
224:            <Split
225:              key={review ? "workspace-with-review" : "workspace-no-review"}
226:              className="flex flex-col gap-0"
227:              direction="vertical"
228:              gutterSize={10}
229:              sizes={review ? [52, 20, 28] : [58, 42]}
230:            >
231:              <section className="overflow-hidden rounded-3xl border border-white/10 bg-[#171d24]">
232:                <div className="flex flex-col gap-3 border-b border-white/10 px-5 py-4 sm:flex-row
sm:items-center sm:justify-between">
233:                  <div>
234:                    <h2 className="text-lg font-semibold text-white">Workspace</h2>
235:                    <p className="text-sm text-[#95a5b4]">Write, run, and submit from the same panel.</p>
236:                  </div>
237:
238:                  <select
239:                    value={language}
240:                    onChange={(e) => setLanguage(e.target.value)}
241:                    className="rounded-xl border border-white/10 bg-[#0f1419] px-3 py-2 text-sm text-white
outline-none"
242:                  >
243:                    <option value="c">C</option>
244:                    <option value="cpp">C++</option>
245:                    <option value="python">Python</option>
246:                    <option value="javascript">JavaScript</option>
247:                    <option value="go">Go</option>
248:                  </select>
249:                </div>
250:
251:                <div className="h-[100%] min-h-[320px]">
252:                  <Editor
253:                    className="h-full"
254:                    theme="vs-dark"
255:                    defaultLanguage="cpp"
256:                    value={code}
257:                    onChange={setCode}
```

```
258:                    options={{
259:                      fontSize: 14,
260:                      minimap: { enabled: false },
261:                      automaticLayout: true,
262:                      scrollBeyondLastLine: false,
263:                      padding: { top: 16 },
264:                    }}
265:                  />
266:                </div>
267:              </section>
268:
269:              <section className="overflow-hidden rounded-3xl border border-white/10 bg-[#171d24]">
270:                <div className="border-b border-white/10 px-5 py-4">
271:                  <h2 className="text-sm font-semibold uppercase tracking-[0.24em] text-[#8fa1b3]">
272:                    Console
273:                  </h2>
274:                </div>
275:
276:                <div className="space-y-4 px-5 py-5">
277:                  <textarea
278:                    value={input}
279:                    onChange={(e) => setInput(e.target.value)}
280:                    rows={4}
281:                    placeholder="Custom input"
282:                    className="w-full rounded-2xl border border-white/10 bg-[#0f1419] p-3 text-sm text-
white outline-none placeholder:text-[#70808f]"
283:                  />
284:
285:                  <div className="flex flex-wrap gap-3">
286:                    <button
287:                      onClick={handleRun}
288:                      disabled={loading}
289:                      className="rounded-full bg-[#d7e2ec] px-5 py-2 text-sm font-medium text-[#10212d]
transition hover:bg-white disabled:cursor-not-allowed disabled:opacity-60"
290:                    >
291:                      {loading ? "Running..." : "Run Code"}
292:                    </button>
293:                    <button
294:                      onClick={handleSubmit}
295:                      disabled={loading}
296:                      className="rounded-full border border-emerald-500/30 bg-emerald-500/10 px-5 py-2
text-sm font-medium text-emerald-200 transition hover:bg-emerald-500/15 disabled:cursor-not-allowed
disabled:opacity-60"
297:                    >
298:                      Submit
299:                    </button>
300:                    <button
301:                      onClick={handleReview}
302:                      disabled={reviewLoading}
303:                      className="rounded-full border border-white/12 bg-[#12171d] px-5 py-2 text-sm font-
medium text-white transition hover:bg-[#141a21] disabled:cursor-not-allowed disabled:opacity-60"
304:                    >
305:                      {reviewLoading ? "Reviewing..." : "AI Review"}
306:                    </button>
307:                  </div>
308:
309:                  <div className="rounded-2xl border border-white/8 bg-[#0f1419] p-4">
310:                    <p className="text-xs uppercase tracking-[0.2em] text-[#70808f]">Output</p>
311:                    <pre className="mt-3 min-h-[96px] whitespace-pre-wrap break-words text-sm leading-6
text-[#dbe4ec]">
312:                      {output || "Run your code to inspect the result here."}
313:                    </pre>
314:                  </div>
315:                </div>
316:              </section>
317:
318:              {review && (
319:                <section className="overflow-y-auto rounded-3xl border border-white/10 bg-[#171d24]">
320:                  <div className="border-b border-white/10 px-5 py-4">
321:                    <h2 className="text-sm font-semibold uppercase tracking-[0.24em] text-[#8fa1b3]">
322:                      Review Notes
```

```
323:                  </h2>
324:                </div>
325:
326:                <div className="px-5 py-5">
327:                  <div className="prose prose-sm prose-invert max-w-none">
328:                    <ReactMarkdown remarkPlugins={[remarkGfm]}>{review}</ReactMarkdown>
329:                  </div>
330:                </div>
331:              </section>
332:            )}
333:          </Split>
334:        </Split>
335:      </section>
336:    </main>
337:  );
338: }
339:
340: export default ProblemPage;
341:
342: function TopStat({ label, value }) {
343:   return (
344:     <div className="rounded-2xl border border-white/8 bg-[#12171d] px-4 py-4">
345:       <p className="text-xl font-semibold text-white">{value}</p>
346:       <p className="mt-1 text-xs uppercase tracking-[0.2em] text-[#728394]">{label}</p>
347:     </div>
348:   );
349: }
350:
351: export const ProblemLoader = async ({ params }) => {
352:   try {
353:     const { problemId } = params;
354:     const res = await axios.get(`${BACKEND_URL}/api/problem/${problemId}`, {
355:       withCredentials: true,
356:     });
357:     return { problem: res.data };
358:   } catch (err) {
359:     if (err.response?.status === 401) {
360:       return redirect("/login");
361:     }
362:     throw new Response("Problem not found", { status: 404 });
363:   }
364: };
```

Line 11: Defines the main coding workspace page.

Lines 12-14: Receives preloaded problem data.

Lines 15-32: Stores editor code state and other local workspace state.

Lines 33-44: Defines lazy submission-history fetch.

Lines 45-50: Opens/closes submission history panel.

Lines 51-71: Runs code through compiler service `/run`.

Lines 72-89: Submits code to backend judge endpoint.

Lines 90-103: Requests AI review from compiler service.

Lines 104-122: Resets starter template when language changes.

Lines 123-341: Renders the full workspace UI.

Lines 342-350: Defines header stat card.

Lines 351-364: Defines route loader for problem detail.

Profile Page

File: Frontend/src/Pages/ProfilePage.jsx

Purpose: Shows the logged-in user's profile summary, stats, and submission history.

How it fits into the app: Rendered at `/profile` inside `RootLayout`; page fetches profile and submissions directly on mount.

Interview-ready talking points:

Uses local state rather than Redux for profile details and submission history.

Logout is driven through Redux thunk, but data loading is page-local.

Implementation notes:

Short variable names like `u`, `su`, `s`, `ss`, `d`, and `n` make the implementation less readable than the rest of the codebase.

Code

```
 1: import { useEffect, useMemo, useState } from "react";
 2: import axios from "axios";
 3: import { useDispatch } from "react-redux";
 4: import { useNavigate } from "react-router-dom";
 5: import { logoutThunk } from "../Store/Features/Auth/authThunks.js";
 6: import SubmissionTable from "../Components/SubmissionTable.jsx";
 7: import MeshPageShell from "../Components/MeshPageShell.jsx";
 8: import { BACKEND_URL } from "../config/urls.js";
 9:
10: function Stat({ label, value }) {
11:   return (
12:     <div className="rounded-2xl border border-white/12 bg-black/20 px-4 py-4 backdrop-blur-sm">
13:       <p className="text-2xl font-semibold text-white">{value}</p>
14:       <p className="mt-1 text-xs uppercase tracking-[0.2em] text-white/55">{label}</p>
15:     </div>
16:   );
17: }
18:
19: export default function ProfilePage() {
20:   const [u, su] = useState(null);
21:   const [s, ss] = useState([]);
22:   const d = useDispatch();
23:   const n = useNavigate();
24:
25:   const logout = async () => {
26:     await d(logoutThunk());
27:     n("/login", { replace: true });
28:   };
29:
30:   useEffect(() => {
31:     (async () => {
32:       try {
33:         const pr = await axios.get(`${BACKEND_URL}/api/profile`, {
34:           withCredentials: true,
35:         });
36:         su(pr.data.user);
37:
38:         const sr = await axios.get(`${BACKEND_URL}/api/submissions`, {
39:           withCredentials: true,
40:         });
41:         ss(sr.data.submissions);
42:       } catch {
43:         n("/login");
44:       }
45:     })();
46:   }, [n]);
47:
48:   const acceptanceRate = useMemo(() => {
49:     if (!u) return "0.0";
50:     return ((u.acceptedSubmissions / Math.max(u.totalSubmissions, 1)) * 100).toFixed(1);
51:   }, [u]);
52:
53:   if (!u) {
```

```
 54:     return (
 55:       <MeshPageShell contentClassName="flex min-h-screen items-center justify-center">
 56:         <div className="rounded-3xl border border-white/15 bg-black/25 px-6 py-5 text-white backdrop-
blur-md">
 57:           Loading...
 58:         </div>
 59:       </MeshPageShell>
 60:     );
 61:   }
 62:
 63:   return (
 64:     <MeshPageShell>
 65:       <div className="grid gap-6 xl:grid-cols-[1.35fr_0.65fr]">
 66:         <section className="rounded-[32px] border border-white/15 bg-black/25 p-6 shadow-
[0_18px_60px_rgba(0,0,0,0.28)] backdrop-blur-md sm:p-8">
 67:           <div className="flex flex-col gap-6 border-b border-white/10 pb-6 lg:flex-row lg:items-end
lg:justify-between">
 68:             <div className="flex items-center gap-5">
 69:               <div className="flex h-24 w-24 items-center justify-center rounded-full border border-
white/15 bg-white/10 text-4xl font-semibold text-white">
 70:                 {u.userName[0].toUpperCase()}
 71:               </div>
 72:
 73:               <div>
 74:                 <p className="text-xs font-semibold uppercase tracking-[0.28em] text-white/60">
 75:                   Profile
 76:                 </p>
 77:                 <h1 className="mt-3 text-3xl font-semibold text-white sm:text-4xl">
 78:                   {u.userName}
 79:                 </h1>
 80:                 <p className="mt-2 text-sm text-white/72">{u.email}</p>
 81:               </div>
 82:             </div>
 83:
 84:             <button
 85:               onClick={logout}
 86:               className="rounded-full border border-rose-400/25 bg-rose-400/10 px-5 py-2.5 text-sm font-
medium text-rose-100 transition hover:bg-rose-400/15"
 87:             >
 88:               Logout
 89:             </button>
 90:           </div>
 91:
 92:           <div className="mt-6 grid gap-3 sm:grid-cols-2 xl:grid-cols-4">
 93:             <Stat label="Solved" value={u.problemsSolved.length} />
 94:             <Stat label="Submissions" value={u.totalSubmissions} />
 95:             <Stat label="Accepted" value={u.acceptedSubmissions} />
 96:             <Stat label="Acceptance" value={`${acceptanceRate}%`} />
 97:           </div>
 98:
 99:           <div className="mt-6 rounded-[28px] border border-white/12 bg-white/[0.05] p-5">
100:             <p className="text-xs font-semibold uppercase tracking-[0.24em] text-white/55">
101:               About
102:             </p>
103:             <p className="mt-4 whitespace-pre-wrap text-sm leading-7 text-white/78">
104:               {u.bio || "No bio added yet."}
105:             </p>
106:           </div>
107:         </section>
108:
109:         <aside className="space-y-6">
110:           <Panel title="Profile Snapshot">
111:             <InfoRow label="Institution" value={u.institution || "-"} />
112:             <InfoRow label="Location" value={u.location || "-"} />
113:             <InfoRow label="Status" value={u.role || "User"} />
114:           </Panel>
115:
116:           <Panel title="Current Focus">
117:             <div className="rounded-2xl border border-white/12 bg-black/20 px-4 py-4 backdrop-blur-sm">
118:               <p className="text-sm font-medium text-white">Keep solving consistently</p>
119:               <p className="mt-2 text-sm leading-6 text-white/70">
```

```
120:                 Your recent submissions and acceptance rate are tracked below.
121:               </p>
122:             </div>
123:           </Panel>
124:         </aside>
125:       </div>
126:
127:       <section className="mt-6 rounded-[32px] border border-white/15 bg-black/25 p-6 shadow-
[0_18px_60px_rgba(0,0,0,0.28)] backdrop-blur-md">
128:         <div className="mb-4 flex flex-col gap-2 sm:flex-row sm:items-center sm:justify-between">
129:           <div>
130:             <h2 className="text-lg font-semibold text-white">Submission History</h2>
131:             <p className="text-sm text-white/68">Recent work across the platform.</p>
132:           </div>
133:         </div>
134:
135:         {!!s.length ? (
136:           <SubmissionTable submissions={s} />
137:         ) : (
138:           <p className="rounded-2xl border border-dashed border-white/15 bg-black/20 px-4 py-8 text-
center text-sm text-white/65">
139:             No submissions yet.
140:           </p>
141:         )}
142:       </section>
143:     </MeshPageShell>
144:   );
145: }
146:
147: function Panel({ title, children }) {
148:   return (
149:     <div className="rounded-[28px] border border-white/15 bg-black/25 p-5 backdrop-blur-md">
150:       <h2 className="text-sm font-semibold uppercase tracking-[0.24em] text-white/58">{title}</h2>
151:       <div className="mt-4 space-y-3">{children}</div>
152:     </div>
153:   );
154: }
155:
156: function InfoRow({ label, value }) {
157:   return (
158:     <div className="flex items-center justify-between rounded-2xl border border-white/12 bg-black/20 px-4
py-4 backdrop-blur-sm">
159:       <span className="text-sm text-white/72">{label}</span>
160:       <span className="text-sm font-medium text-white">{value}</span>
161:     </div>
162:   );
163: }
```

Line 19: Defines profile page.

Lines 20-24: Stores user profile locally.

Lines 25-29: Logs out through Redux and redirects.

Lines 30-47: Fetches profile and submissions on mount.

Lines 48-52: Computes acceptance rate from loaded profile data.

Lines 53-10: Shows loading state until profile arrives.

Lines 11-146: Renders profile layout.

Lines 147-155: Defines reusable side panel.

Lines 156-163: Defines metadata row.

Register Page

File: Frontend/src/Pages/RegisterPage.jsx

Purpose: Implements user registration UI with local password-strength validation and backend submission.

How it fits into the app: Rendered at `/register` inside `LoginLayout`.

Interview-ready talking points:

Runs client-side password complexity check before calling backend.

Sends `{ role: 'user' }` to backend register endpoint.

Implementation notes:

Backend still re-validates all fields, which is correct defense in depth.

Code

```
 1: import { useEffect, useState } from "react";
 2: import { useNavigate, Link } from "react-router-dom";
 3: import axios from "axios";
 4: import mesh from "../assets/mesh.png";
 5: import { BACKEND_URL } from "../config/urls.js";
 6:
 7: export default function RegisterPage() {
 8:   const navigate = useNavigate();
 9:   const [formFields, setFormFields] = useState({ userName: "", email: "", password: "" });
10:   const [submit, setSubmit] = useState(false);
11:   const [formError, setFormError] = useState("");
12:   const [hasSubmitted, setHasSubmitted] = useState(false);
13:   const [showPassword, setShowPassword] = useState(false);
14:
15:   const passwordRequirements =
16:     "Use 8-32 characters with uppercase, lowercase, number, and special character.";
17:
18:   const isStrongPassword = (password) =>
19:     /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^A-Za-z0-9]).{8,32}$/.test(password);
20:
21:   useEffect(() => {
22:     if (!submit) return;
23:     setHasSubmitted(true);
24:     const { userName, email, password } = formFields;
25:     if (!userName || !email || !password) {
26:       setFormError("Please fill all the required fields.");
27:       setSubmit(false);
28:       return;
29:     }
30:     if (!isStrongPassword(password)) {
31:       setFormError(passwordRequirements);
32:       setSubmit(false);
33:       return;
34:     }
35:     (async () => {
36:       try {
37:         await axios.post(
38:           `${BACKEND_URL}/api/auth/register`,
39:           { ...formFields, role: "user" },
40:           { withCredentials: true }
41:         );
42:         setFormError("");
43:         navigate("/login");
44:       } catch (err) {
45:         console.error("Register error", err.response?.data || err.message);
46:         const apiErrors = err?.response?.data?.errors;
47:         if (Array.isArray(apiErrors) && apiErrors.length) {
48:           setFormError(apiErrors[0].msg);
49:         } else {
50:           setFormError(err?.response?.data?.message || "Registration failed.");
51:         }
52:       }
53:       setSubmit(false);
54:     })();
```

```
 55:   }, [submit, formFields, navigate]);
 56:
 57:   return (
 58:     <main className="relative min-h-screen overflow-hidden bg-[#0a0f14] text-white">
 59:       <img
 60:         src={mesh}
 61:         alt="Code Bharat background"
 62:         className="absolute inset-0 h-full w-full object-cover object-center"
 63:       />
 64:       <div className="absolute inset-0 bg-[linear-
gradient(90deg,rgba(6,10,14,0.72)_0%,rgba(6,10,14,0.46)_45%,rgba(6,10,14,0.38)_100%)]" />
 65:       <div className="absolute inset-0 bg-[radial-
gradient(circle_at_top_right,rgba(255,255,255,0.12),transparent_32%),radial-
gradient(circle_at_bottom_left,rgba(255,255,255,0.08),transparent_28%)]" />
 66:
 67:       <section className="relative mx-auto flex min-h-screen max-w-7xl items-center justify-end px-4 py-
10 sm:px-6 lg:px-8">
 68:         <div className="w-full max-w-md rounded-[30px] border border-white/15 bg-black/25 p-7 backdrop-
blur-md sm:p-8">
 69:           <p className="text-xs font-semibold uppercase tracking-[0.32em] text-white/70">
 70:             Register
 71:           </p>
 72:           <h1 className="mt-4 text-3xl font-semibold text-white">Create account</h1>
 73:           <p className="mt-2 text-sm text-white/75">Start solving.</p>
 74:
 75:           <form className="mt-8 space-y-5" onSubmit={(e) => e.preventDefault()}>
 76:             <div>
 77:               <label htmlFor="userName" className="mb-2 block text-sm font-medium text-white/85">
 78:                 User Name
 79:               </label>
 80:               <input
 81:                 id="userName"
 82:                 name="userName"
 83:                 type="text"
 84:                 className="w-full rounded-2xl border border-white/15 bg-black/25 px-4 py-3 text-white
outline-none placeholder:text-white/35"
 85:                 onChange={(e) => setFormFields((p) => ({ ...p, userName: e.target.value }))}
 86:               />
 87:             </div>
 88:
 89:             <div>
 90:               <label htmlFor="email" className="mb-2 block text-sm font-medium text-white/85">
 91:                 Email
 92:               </label>
 93:               <input
 94:                 id="email"
 95:                 name="email"
 96:                 type="email"
 97:                 className="w-full rounded-2xl border border-white/15 bg-black/25 px-4 py-3 text-white
outline-none placeholder:text-white/35"
 98:                 onChange={(e) => setFormFields((p) => ({ ...p, email: e.target.value }))}
 99:               />
100:             </div>
101:
102:             <div>
103:               <label htmlFor="password" className="mb-2 block text-sm font-medium text-white/85">
104:                 Password
105:               </label>
106:               <div className="relative">
107:                 <input
108:                   id="password"
109:                   name="password"
110:                   type={showPassword ? "text" : "password"}
111:                   minLength={8}
112:                   className="w-full rounded-2xl border border-white/15 bg-black/25 px-4 py-3 pr-16 text-
white outline-none placeholder:text-white/35"
113:                   onChange={(e) => setFormFields((p) => ({ ...p, password: e.target.value }))}
114:                 />
115:                 <button
116:                   type="button"
117:                   onClick={() => setShowPassword((current) => !current)}
```

```
118:                    className="absolute right-4 top-1/2 -translate-y-1/2 text-xs font-medium text-white/70
transition hover:text-white"
119:                  >
120:                    {showPassword ? "Hide" : "Show"}
121:                  </button>
122:                </div>
123:                <p className="mt-2 text-xs text-white/65">{passwordRequirements}</p>
124:              </div>
125:
126:              {hasSubmitted && formError && <p className="text-sm text-rose-200">{formError}</p>}
127:
128:              <button
129:                type="submit"
130:                onClick={() => setSubmit(true)}
131:                className="w-full rounded-full bg-white px-6 py-3 text-sm font-medium text-[#10212d]
transition hover:bg-[#e7eef5]"
132:              >
133:                Create Account
134:              </button>
135:            </form>
136:
137:            <p className="mt-6 text-sm text-white/72">
138:              Already have an account?{" "}
139:              <Link to="/login" className="font-medium text-white">
140:                Log in
141:              </Link>
142:            </p>
143:          </div>
144:        </section>
145:      </main>
146:    );
147: }
```

Lines 7-8: Defines register page.

Lines 9-14: Stores registration form data.

Lines 15-17: Defines visible password guidance text.

Lines 18-20: Defines client-side password rule.

Lines 21-36: Runs submit-side effect when submit flag changes.

Lines 37-56: Calls backend registration endpoint.

Lines 57-147: Renders registration form UI.

Auth Navbar

File: Frontend/src/Components/AuthNavbar.jsx

Purpose: Minimal navbar used on login/register screens.

How it fits into the app: Rendered by `LoginLayout` above auth pages.

Interview-ready talking points:

Keeps auth screens visually simple and separate from the full main navbar.

Code

```
 1: import { Link } from "react-router-dom";
 2: import logo from "../assets/logo.png";
 3:
 4: export default function AuthNavbar() {
 5:   return (
 6:     <nav className="sticky top-0 z-50 bg-gradient-to-r from-[#0e0e0e] via-[#1a1a1a] to-[#0e0e0e] text-
[#e5e5e5] shadow-xl">
 7:       <div className="mx-auto flex h-20 max-w-7xl items-center justify-between px-6">
 8:         <Link to="/" className="flex items-center gap-3">
 9:           <img src={logo} alt="Code Bharat logo" className="h-10 w-auto" />
10:         </Link>
11:         <Link
12:           to="/"
13:           className="rounded-full bg-gradient-to-r from-gray-500 to-gray-600 px-4 py-1.5 text-sm font-
medium transition-colors hover:from-gray-400 hover:to-gray-500"
14:         >
15:           &larr; Home
16:         </Link>
17:       </div>
18:     </nav>
19:   );
20: }
```

Lines 4-5: Defines auth-specific navbar.

Lines 6-7: Creates the top gradient navigation shell.

Lines 8-7: Provides brand link to home.

Lines 8-20: Provides a direct way back to home.

```
Footer
File: Frontend/src/Components/Footer.jsx
Purpose: Shared footer shown on layouts that include it.
How it fits into the app: Rendered by `RootLayout` and `LoginLayout`.
Interview-ready talking points:
Uses simple external links and dynamic current year.
```

Code

```
 1: export default function Footer() {
 2:   return (
 3:     <footer className="mt-16 bg-gradient-to-r from-[#0f0f0f] via-[#1a1a1a] to-[#0f0f0f] py-6 text-gray-
400">
 4:       <div className="mx-auto flex max-w-6xl flex-col items-center justify-between px-4 text-sm md:flex-
row">
 5:         <p>&copy; {new Date().getFullYear()} Code Bharat. All rights reserved.</p>
 6:         <div className="mt-4 flex gap-6 md:mt-0">
 7:           <a
 8:             href="https://github.com/Harsh3185/Code-Bharat"
 9:             target="_blank"
10:             rel="noopener noreferrer"
11:             className="hover:text-white"
12:           >
13:             GitHub
14:           </a>
15:           <a href="https://harsh-choudhary-nine.vercel.app/" className="hover:text-white">
16:             About
17:           </a>
18:           <a href="https://harsh-choudhary-nine.vercel.app/" className="hover:text-white">
19:             Contact
20:           </a>
21:         </div>
22:       </div>
23:     </footer>
24:   );
25: }
```

```
Lines 1-2: Defines the shared footer component.
Lines 3-4: Creates the footer container.
Lines 5-7: Computes current year dynamically.
Lines 8-25: Links to project GitHub.
```

Mesh Page Shell

File: Frontend/src/Components/MeshPageShell.jsx

Purpose: Reusable page shell that applies the mesh background and atmospheric overlays.

How it fits into the app: Used by several public pages like Explore, Contests, Leaderboard, Problem Set, and Profile.

Interview-ready talking points:

Centralizes the signature visual background so pages stay consistent.

Accepts `contentClassName` for small layout variation without duplicating shell code.

Code

```
 1: import mesh from "../assets/mesh.png";
 2:
 3: export default function MeshPageShell({ children, contentClassName = "" }) {
 4:   return (
 5:     <main className="relative min-h-screen overflow-hidden bg-[#0a0f14] text-white">
 6:       <img
 7:         src={mesh}
 8:         alt="Code Bharat background"
 9:         className="absolute inset-0 h-full w-full object-cover object-center"
10:       />
11:       <div className="absolute inset-0 bg-[linear-
gradient(90deg,rgba(6,10,14,0.76)_0%,rgba(6,10,14,0.5)_45%,rgba(6,10,14,0.42)_100%)]" />
12:       <div className="absolute inset-0 bg-[radial-
gradient(circle_at_top_right,rgba(255,255,255,0.12),transparent_32%),radial-
gradient(circle_at_bottom_left,rgba(255,255,255,0.08),transparent_28%)]" />
13:
14:       <section className={`relative mx-auto max-w-7xl px-4 py-8 sm:px-6 lg:px-8 ${contentClassName}`}>
15:         {children}
16:       </section>
17:     </main>
18:   );
19: }
```

Lines 3-4: Defines the reusable shell component.

Line 5: Creates the background canvas.

Lines 6-13: Places the mesh image as full-page background.

Lines 14-19: Wraps page content in a centered max-width container.

Main Navbar

File: Frontend/src/Components/Navbar.jsx

Purpose: Primary navigation for the public app, with auth-aware rendering and responsive mobile menu behavior.

How it fits into the app: Rendered by `RootLayout` and `ProblemLayout` before page content.

Interview-ready talking points:

Hydrates auth state by dispatching `fetchUser` when needed.

Shows admin shortcut only for admin users.

Desktop and mobile nav behavior are both handled in one component.

Implementation notes:

Navbar is a practical place where global Redux auth state is consumed.

Because RootLayout renders Navbar before HomePage, auth state often hydrates there first.

Code

```
  1: import { useState, useEffect } from 'react';
  2: import { Link, useLocation } from 'react-router-dom';
  3: import { useDispatch, useSelector } from 'react-redux';
  4:
  5: import logo from '../assets/logo.png';
  6: import { fetchUser } from '../Store/Features/Auth/authThunks.js';
  7:
  8: export default function Navbar() {
  9:   const [open, setOpen] = useState(false);
 10:   const { pathname } = useLocation();
 11:   const dispatch = useDispatch();
 12:   const { user } = useSelector((s) => s.auth);
 13:   const isAdmin = user?.role?.toLowerCase() === 'admin';
 14:
 15:   useEffect(() => {
 16:     if (!user) dispatch(fetchUser());
 17:   }, [dispatch, user]);
 18:
 19:   const nav = [
 20:     { path: '/explore', label: 'Explore' },
 21:     { path: '/problems', label: 'Problems' },
 22:     { path: '/contests', label: 'Contests' },
 23:     { path: '/leaderboard', label: 'Leaderboard' }
 24:   ];
 25:
 26:   return (
 27:     <nav className="sticky top-0 z-50 w-full backdrop-blur bg-[#0e0e0ee6] border-b border-white/10
shadow-lg">
 28:       <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
 29:         <div className="flex items-center justify-between h-20">
 30:           <Link to="/" className="flex items-center gap-3 group">
 31:             <img
 32:               src={logo}
 33:               alt="Code Bharat logo"
 34:               className="h-12 w-auto transition-transform duration-300 group-hover:scale-105"
 35:             />
 36:           </Link>
 37:
 38:           <div className="hidden md:flex items-center gap-8 text-sm font-medium">
 39:             {nav.map(({ path, label }) => (
 40:               <Link
 41:                 key={path}
 42:                 to={path}
 43:                 className={`relative px-3 py-2 duration-300 ${
 44:                   pathname.startsWith(path) ? 'text-gray-50' : 'text-gray-300 hover:text-gray-50'
 45:                 }`}
 46:               >
 47:                 {label}
 48:                 <span
 49:                   className={`absolute left-0 -bottom-0.5 h-0.5 w-full bg-gradient-to-r from-gray-300 to-
gray-500 origin-left
```

```
 50:                   transform ${
 51:                     pathname.startsWith(path) ? 'scale-x-100' : 'scale-x-0 group-hover:scale-x-100'
 52:                   } duration-300`}
 53:                 />
 54:               </Link>
 55:             ))}
 56:
 57:             {user ? (
 58:               <>
 59:                 {isAdmin && (
 60:                   <Link
 61:                     to="/admin"
 62:                     className="rounded-full border border-white/10 px-4 py-2 text-gray-200 transition
hover:border-white/20 hover:bg-white/5 hover:text-white"
 63:                   >
 64:                     Admin
 65:                   </Link>
 66:                 )}
 67:                 <Link to="/profile">
 68:                   <div className="ml-1 flex h-10 w-10 items-center justify-center rounded-full bg-blue-
600 text-lg font-bold uppercase text-white hover:bg-blue-700">
 69:                     {user.userName?.[0] ?? 'U'}
 70:                   </div>
 71:                 </Link>
 72:               </>
 73:             ) : (
 74:               <Link
 75:                 to="/login"
 76:                 className="ml-4 px-6 py-2 rounded-full bg-gradient-to-r from-gray-600 to-gray-800
hover:from-gray-500 hover:to-gray-700 text-white font-semibold shadow-md"
 77:               >
 78:                 Login
 79:               </Link>
 80:             )}
 81:           </div>
 82:
 83:           <button
 84:             onClick={() => setOpen(!open)}
 85:             className="md:hidden w-9 h-9 flex flex-col justify-center items-center"
 86:             aria-label="Toggle menu"
 87:           >
 88:             <span
 89:               className={`block w-6 h-0.5 bg-gray-200 transition-transform ${
 90:                 open ? 'rotate-45 translate-y-1' : ''
 91:               }`}
 92:             />
 93:             <span
 94:               className={`block w-6 h-0.5 bg-gray-200 mt-1 transition-opacity ${
 95:                 open ? 'opacity-0' : 'opacity-100'
 96:               }`}
 97:             />
 98:             <span
 99:               className={`block w-6 h-0.5 bg-gray-200 mt-1 transition-transform ${
100:                 open ? '-rotate-45 -translate-y-1' : ''
101:               }`}
102:             />
103:           </button>
104:         </div>
105:
106:         <div
107:           className={`md:hidden grid transition-[grid-template-rows] duration-300 overflow-hidden ${
108:             open ? 'grid-rows-[1fr] pb-4' : 'grid-rows-[0fr]'
109:           }`}
110:         >
111:           <div className="overflow-hidden flex flex-col gap-3 text-sm font-medium border-t border-
white/10 pt-4">
112:             {nav.map(({ path, label }) => (
113:               <Link
114:                 key={path}
115:                 to={path}
116:                 onClick={() => setOpen(false)}
```

```
117:                 className="px-4 py-2 text-gray-200/90 hover:bg-white/10 rounded-lg"
118:               >
119:                 {label}
120:               </Link>
121:             ))}
122:
123:             {user ? (
124:               <>
125:                 {isAdmin && (
126:                   <Link
127:                     to="/admin"
128:                     onClick={() => setOpen(false)}
129:                     className="mx-4 rounded-lg border border-white/10 px-4 py-2 text-gray-200 transition
hover:bg-white/10"
130:                   >
131:                     Admin Console
132:                   </Link>
133:                 )}
134:                 <Link to="/profile" onClick={() => setOpen(false)}>
135:                   <div className="ml-4 mt-2 flex h-10 w-10 items-center justify-center rounded-full bg-
blue-600 text-lg font-bold uppercase text-white hover:bg-blue-700">
136:                     {user.userName?.[0] ?? 'U'}
137:                   </div>
138:                 </Link>
139:               </>
140:             ) : (
141:               <Link
142:                 to="/login"
143:                 onClick={() => setOpen(false)}
144:                 className="mx-4 mt-2 px-4 py-2 text-center rounded-lg bg-gradient-to-r from-gray-600 to-
gray-800 text-white font-semibold hover:from-gray-500 hover:to-gray-700"
145:               >
146:                 Login
147:               </Link>
148:             )}
149:           </div>
150:         </div>
151:       </div>
152:     </nav>
153:   );
154: }
```

Line 8: Defines the main navbar component.

Line 9: Stores mobile menu open state.

Lines 10-11: Uses current URL to highlight active nav items.

Lines 12-14: Reads auth state from Redux.

Lines 15-18: Fetches current user if auth state is empty.

Lines 19-25: Defines top-level navigation links.

Lines 26-56: Starts rendering navbar UI.

Lines 57-82: Branches UI based on logged-in vs logged-out state.

Lines 83-154: Defines mobile hamburger toggle.

Submission Table

File: Frontend/src/Components/SubmissionTable.jsx

Purpose: Shared table for displaying submission history.

How it fits into the app: Used on profile page and problem page submission history panel.

Interview-ready talking points:

Centralizing the table avoids repeating status rendering and date formatting logic.

Links submission rows back to the problem detail page.

Code

```
 1: import { Link } from "react-router-dom";
 2:
 3: export default function SubmissionTable({ submissions = [] }) {
 4:   if (!submissions.length) return null;
 5:
 6:   return (
 7:     <div className="overflow-x-auto rounded-2xl border border-white/8">
 8:       <table className="min-w-full text-sm">
 9:         <thead className="bg-[#12171d] text-left text-xs uppercase tracking-[0.2em] text-[#748597]">
10:           <tr>
11:             <th className="px-5 py-4 font-medium">#</th>
12:             <th className="px-5 py-4 font-medium">Problem</th>
13:             <th className="px-5 py-4 font-medium">Language</th>
14:             <th className="px-5 py-4 font-medium">Status</th>
15:             <th className="px-5 py-4 font-medium">Submitted</th>
16:           </tr>
17:         </thead>
18:         <tbody className="divide-y divide-white/6">
19:           {submissions.map((submission, index) => (
20:             <tr key={submission._id} className="hover:bg-white/[0.03]">
21:               <td className="px-5 py-4 font-mono text-[#b7c2cc]">{index + 1}</td>
22:               <td className="px-5 py-4">
23:                 <Link
24:                   to={`/problems/${submission.problemId}`}
25:                   className="font-medium text-white transition hover:text-[#dbe6ef]"
26:                 >
27:                   #
28:                   {submission.problemNumber != null
29:                     ? String(submission.problemNumber).padStart(3, "0")
30:                     : "???"}
31:                 </Link>
32:               </td>
33:               <td className="px-5 py-4 text-[#d2dce5]">{submission.language}</td>
34:               <td className="px-5 py-4">
35:                 <span className={`rounded-full px-3 py-1 text-xs font-semibold
${statusClasses(submission.status)}`}>
36:                   {submission.status}
37:                 </span>
38:               </td>
39:               <td className="px-5 py-4 text-[#9aabb9]">
40:                 {new Date(submission.createdAt).toLocaleString()}
41:               </td>
42:             </tr>
43:           ))}
44:         </tbody>
45:       </table>
46:     </div>
47:   );
48: }
49:
50: function statusClasses(status) {
51:   if (status === "Accepted") return "bg-emerald-500/15 text-emerald-200";
52:   if (status.includes("Wrong")) return "bg-amber-500/15 text-amber-200";
53:   return "bg-rose-500/15 text-rose-200";
54: }
```

```
Line 3: Defines shared submission table component.
Lines 4-7: Skips rendering when there is no data.
Lines 8-18: Builds table markup.
Lines 19-49: Renders one row per submission.
Lines 50-54: Maps statuses to badge color classes.
```

Admin Auth Layout

File: Frontend/src/Layouts/AdminAuthLayout.jsx

Purpose: Simple wrapper layout for admin login and admin registration screens.

How it fits into the app: Applied to `/admin/login` and `/admin/register` routes.

Interview-ready talking points:

Keeps admin auth pages separate from the public root/login layouts.

Code

```
1: import { Outlet } from "react-router-dom";
2:
3: export default function AdminAuthLayout() {
4:   return (
5:     <div className="min-h-screen bg-[#0d1117] text-white">
6:       <Outlet />
7:     </div>
8:   );
9: }
```

Lines 3-5: Defines the admin auth layout.

Lines 6-9: Renders the nested admin auth page.

Admin Layout

File: Frontend/src/Layouts/AdminLayout.jsx

Purpose: Provides the protected admin workspace shell with sidebar navigation and logout action.

How it fits into the app: Wraps admin dashboard and admin problem-management pages after `RequireAdmin` passes.

Interview-ready talking points:

Sidebar layout clearly separates admin workspace from user-facing UI.

Logout is driven through Redux thunk and redirects to admin login.

Implementation notes:

The `Open main site` link uses plain `<a href="/">`, which triggers a full page reload instead of client-side navigation.

Code

```
 1: import { Outlet, NavLink, useNavigate } from "react-router-dom";
 2: import { useDispatch, useSelector } from "react-redux";
 3: import { logoutThunk } from "../Store/Features/Auth/authThunks.js";
 4: import mesh from "../assets/mesh.png";
 5:
 6: const links = [
 7:   { to: "/admin", label: "Dashboard", end: true },
 8:   { to: "/admin/problems", label: "Manage Problems" },
 9:   { to: "/admin/problems/new", label: "Add Problem" },
10: ];
11:
12: export default function AdminLayout() {
13:   const dispatch = useDispatch();
14:   const navigate = useNavigate();
15:   const { user } = useSelector((state) => state.auth);
16:
17:   const handleLogout = async () => {
18:     await dispatch(logoutThunk());
19:     navigate("/admin/login", { replace: true });
20:   };
21:
22:   return (
23:     <div className="relative min-h-screen overflow-hidden bg-[#0a0f14] text-[#e8edf2]">
24:       <img
25:         src={mesh}
26:         alt="Code Bharat background"
27:         className="absolute inset-0 h-full w-full object-cover object-center"
28:       />
29:       <div className="absolute inset-0 bg-[linear-
gradient(90deg,rgba(6,10,14,0.78)_0%,rgba(6,10,14,0.56)_45%,rgba(6,10,14,0.48)_100%)]" />
30:       <div className="absolute inset-0 bg-[radial-
gradient(circle_at_top_right,rgba(255,255,255,0.12),transparent_32%),radial-
gradient(circle_at_bottom_left,rgba(255,255,255,0.08),transparent_28%)]" />
31:
32:       <div className="relative mx-auto grid min-h-screen max-w-[1600px] gap-6 px-4 py-6 lg:grid-cols-
[260px_1fr] lg:px-6">
33:         <aside className="rounded-[30px] border border-white/15 bg-black/25 px-5 py-6 backdrop-blur-md">
34:           <div className="border-b border-white/8 pb-6">
35:             <p className="text-xs font-semibold uppercase tracking-[0.28em] text-[#8ea2b6]">
36:               Admin Console
37:             </p>
38:             <h1 className="mt-3 text-2xl font-semibold text-white">Code Bharat</h1>
39:             <p className="mt-2 text-sm text-[#91a1b1]">
40:               Signed in as {user?.userName || "Admin"}
41:             </p>
42:           </div>
43:
44:           <nav className="mt-6 space-y-2">
45:             {links.map((link) => (
46:               <NavLink
47:                 key={link.to}
48:                 to={link.to}
49:                 end={link.end}
```

```
50:                 className={({ isActive }) =>
51:                   `block rounded-2xl px-4 py-3 text-sm transition ${
52:                     isActive
53:                       ? "bg-white text-[#10212d]"
54:                       : "bg-transparent text-[#c7d2dc] hover:bg-white/10"
55:                   }`
56:                 }
57:               >
58:                 {link.label}
59:               </NavLink>
60:             ))}
61:           </nav>
62:
63:           <div className="mt-8 rounded-2xl border border-white/12 bg-black/20 p-4">
64:             <p className="text-xs uppercase tracking-[0.22em] text-[#718293]">Public App</p>
65:             <a href="/" className="mt-2 inline-block text-sm text-white hover:text-[#dbe7f1]">
66:               Open main site
67:             </a>
68:           </div>
69:
70:           <button
71:             onClick={handleLogout}
72:             className="mt-6 w-full rounded-2xl border border-rose-500/30 bg-rose-500/10 px-4 py-3 text-sm
font-medium text-rose-200 transition hover:bg-rose-500/15"
73:           >
74:             Logout
75:           </button>
76:         </aside>
77:
78:         <main className="min-h-screen">
79:           <Outlet />
80:         </main>
81:       </div>
82:     </div>
83:   );
84: }
```

Lines 6-11: Defines sidebar navigation items.

Lines 12-14: Defines the admin shell.

Lines 15-16: Reads current admin user from Redux.

Lines 17-32: Logs out and redirects.

Lines 33-45: Creates the admin sidebar.

Lines 46-78: Renders active-aware sidebar navigation.

Lines 79-84: Renders nested admin page content.

Login Layout

File: Frontend/src/Layouts/LoginLayout.jsx

Purpose: Wraps user login and register pages with auth navbar and footer.

How it fits into the app: Applied to `/login` and `/register` routes.

Interview-ready talking points:

Auth pages reuse the same top and bottom chrome while keeping forms page-specific.

Code

```
 1: import React from 'react'
 2: import { Outlet } from 'react-router-dom'
 3: import AuthNavbar from '../Components/AuthNavbar'
 4: import Footer from '../Components/Footer'
 5:
 6: function LoginLayout() {
 7:     return (
 8:         <div className="min-h-screen bg-[#0f0f0f] text-white flex flex-col">
 9:             <AuthNavbar />
10:             <Outlet />
11:             <Footer />
12:         </div>
13:     )
14: }
15:
16: export default LoginLayout
```

Lines 6-8: Defines login/register layout.

Line 9: Places simplified auth navbar above form pages.

Line 10: Renders nested auth page.

Lines 11-16: Adds footer below auth page.

Problem Layout

File: Frontend/src/Layouts/ProblemLayout.jsx

Purpose: Wraps the problem list and coding workspace with the main navbar.

How it fits into the app: Applied to `/problems` and `/problems/:problemId`.

Interview-ready talking points:

Problem area has its own layout so workspace pages keep a consistent frame without the footer.

Code

```
 1: import React from 'react'
 2: import Navbar from '../Components/Navbar.jsx'
 3: import { Outlet } from 'react-router-dom'
 4:
 5: function ProblemLayout() {
 6:     return (
 7:         <div className="min-h-screen flex flex-col bg-[#0f0f0f] text-white">
 8:             <Navbar />
 9:
10:             <main className="flex-1">
11:                 <Outlet />
12:             </main>
13:
14:         </div>
15:     )
16: }
17:
18: export default ProblemLayout
```

Lines 5-7: Defines problem-area layout.

Lines 8-10: Renders main navbar.

Lines 11-18: Renders nested problem page content.

Root Layout

File: Frontend/src/Layouts/RootLayout.jsx

Purpose: Main public-site layout with navbar, routed content, and footer.

How it fits into the app: Applied to home, profile, contests, leaderboard, and explore routes.

Interview-ready talking points:

Provides consistent shell for the public site.

Code

```
 1: import React from 'react'
 2: import Navbar from '../Components/Navbar.jsx'
 3: import { Outlet } from 'react-router-dom'
 4: import Footer from '../Components/Footer.jsx'
 5:
 6: function RootLayout() {
 7:   return (
 8:     <div className="min-h-screen flex flex-col bg-[#0f0f0f] text-white">
 9:       <Navbar />
10:
11:       <main className="flex-1">
12:         <Outlet />
13:       </main>
14:       <Footer />
15:     </div>
16:   )
17: }
18:
19: export default RootLayout
```

Lines 6-8: Defines main public layout.

Lines 9-11: Renders shared top navigation.

Lines 12-13: Renders nested public page.

Lines 14-19: Renders shared footer.

Require Admin Guard

File: Frontend/src/Middleware/RequireAdmin.jsx

Purpose: Protects admin routes by fetching current user and redirecting non-admins.

How it fits into the app: Runs before `AdminLayout` and all nested admin pages.

Interview-ready talking points:

Acts like client-side route middleware using React Router `<Navigate />`.

Relies on Redux auth state and `fetchUser` thunk.

Implementation notes:

This is frontend route protection; real security still depends on backend auth and admin checks.

Code

```
 1: import { useEffect } from "react";
 2: import { useDispatch, useSelector } from "react-redux";
 3: import { Navigate, Outlet } from "react-router-dom";
 4: import { fetchUser } from "../Store/Features/Auth/authThunks.js";
 5:
 6: export default function RequireAdmin() {
 7:   const dispatch = useDispatch();
 8:   const { user, loading } = useSelector((state) => state.auth);
 9:
10:   useEffect(() => {
11:     if (!user) {
12:       dispatch(fetchUser());
13:     }
14:   }, [dispatch, user]);
15:
16:   if (loading) {
17:     return (
18:       <div className="flex min-h-screen items-center justify-center bg-[#0f1419] text-white">
19:         Loading...
20:       </div>
21:     );
22:   }
23:
24:   if (!user || user.role?.toLowerCase() !== "admin") {
25:     return <Navigate to="/admin/login" replace />;
26:   }
27:
28:   return <Outlet />;
29: }
```

Lines 6-7: Defines the admin route guard.

Lines 8-9: Reads auth state from Redux.

Lines 10-15: Fetches current user if not already loaded.

Lines 16-23: Shows loading screen while auth is resolving.

Lines 24-27: Redirects non-admin users.

Lines 28-29: Allows admin route tree to render.

```
How To Explain This Frontend In Interviews
Start with the stack: React + Vite + Tailwind + Redux Toolkit + React Router + axios.
Explain route hierarchy from `App.jsx`, then explain what each layout contributes.
Describe auth as cookie-based and shared through Redux auth state.
Explain that most UI pages fetch their own data with axios and local state.
Use `ProblemPage.jsx` as the strongest example because it combines loaders, editor UI, compiler calls,
submission calls, markdown rendering, split panes, and submission history.
Use admin pages as an example of separating privileged flows from normal user flows.
```